AI/MCP 통합 관계형 데이터베이스 관리 시스템 (RuSQL)

프로젝트 통합 문서 | v29 | 김규현, 빌궁 | 2026년

1. 서론
    1.1 개발 목표
    1.2 개발 내용
    1.3 용어 정의
2. 기능적 요구사항
    2.1 SQL 파싱, 실행, DDL, DML
    2.2 함수, 데이터 타입, 제약조건
    2.3 트랜잭션, 스토리지, 서버, 프로토콜
    2.4 클라이언트, UI, AI
    2.5 DCL, 사용자 관리, 역할, 동의어
3. 비기능적 요구사항
    3.1 성능 요구사항
    3.2 신뢰성 요구사항
    3.3 보안 요구사항
    3.4 사용성 요구사항
    3.5 이식성 요구사항
    3.6 호환성 요구사항
4. 사용자 인터페이스 (UI) 명세
5. 기능적 모델링: Use-Case Diagram
6. 구조적 모델링: Class Diagram
7. 클래스 정의
8. 주요 Use-Case 시퀀스 다이어그램

1. 서론

1.1 개발 목표

이 작품은 관계형 데이터베이스 엔진(SQL 처리, 저장, 트랜잭션/동시성 제어, 서버 등)을 구현하고, 이를 다루는 데스크톱 관리 도구와 AI/MCP 기술의 연동 기능을 갖춘 MySQL 프로토콜과 호환되는 관계형 데이터베이스 시스템이다.

1.1.1 개발 배경

관계형 데이터베이스는 대부분의 서비스가 데이터를 저장하고 꺼내 쓰는 기반 소프트웨어이다. 그러나 사용자는 SQL을 입력하고 결과를 받을 뿐, 그 문장이 내부에서 어떻게 해석되고 실행되는지, 장애가 나도 데이터가 어떻게 지켜지는지는 보기 어렵다. 상용 데이터베이스는 소스 규모가 커서 직접 읽으며 배우기에도 부담이 크다. 이에 본 작품은 SQL 파서와 비용 기반 실행 계획, 트랜잭션과 동시성 제어, 로그 기반 복구를 직접 구현하여 관계형 데이터베이스의 핵심 원리를 하나의 시스템 안에서 확인할 수 있게 한다.

직접 만든 엔진이 실제로 쓰이려면 기존 도구와 함께 쓸 수 있어야 한다. 그래서 MySQL 프로토콜을 구현해 mysql 클라이언트와 DBeaver 같은 도구가 별도 드라이버 없이 접속할 수 있게 하고, 같은 SQL을 실제 MySQL에도 보내 결과를 비교하며 정확성을 확인한다.

데이터베이스를 다루는 일에는 SQL 편집, 스키마 확인, 실행 계획 분석, 서버 상태 점검처럼 화면이 도움이 되는 작업이 많다. 이를 위해 데스크톱 관리 도구를 함께 개발하여 에디터와 ERD, 실행 계획 그림, 세션 · 락 모니터링을 한곳에서 쓸 수 있게 한다.

최근에는 AI 어시스턴트에게 자연어로 데이터를 묻고 고치게 하는 사용이 늘고 있다. 하지만 AI가 SQL을 마음대로 실행하면 DROP처럼 되돌릴 수 없는 문장이 확인 없이 실행될 위험이 있다. 본 작품은 개방형 표준인 MCP로 Claude Desktop과 연결하여 AI가 데이터베이스를 다루게 하고, 위험한 문장은 사용자가 확인해야만 실행되도록 안전장치를 둔다. AI 추론은 Claude Desktop이 맡으므로 API 키가 필요 없다.

개발은 1학기에 Rust로 만든 프로토타입으로 시작했고, 2학기에는 엔진을 C++로 전면 재작성하면서 기능을 늘리는 데 그치지 않고 정합성과 동시성, 성능을 깊이 다듬는 데 무게를 두었다.

1.1.2 개발 환경

표 1-1은 작품의 기본 정보와 책임자 · 참여 학생을, 표 1-2는 개발과 시험에 사용한 소프트웨어 환경을 정리한다.

표 1-1. 작품 개요

항목	내용
작품명 (국문)	AI/MCP 통합 관계형 데이터베이스 관리 시스템 (RuSQL)
작품명 (영문)	AI/MCP Integrated Relational Database Management System (RuSQL)
프로젝트 유형	졸업 작품 (충북대학교 소프트웨어학과 졸업작품연구과제)
책임자	김규현 (충북대학교 소프트웨어학과, 학번 2021041017)
참여학생	김규현 (2021041017, 소프트웨어학과), Namsraijalbuu Bilguuntuguldur (2023041082, 소프트웨어학부)
지도교수	이종연

표 1-2. 소프트웨어 개발 환경

구분	명칭	버전	용도
운영체제	Windows 11 Pro (64비트)	10.0.26200	개발 및 시험 환경
핵심 개발 언어	C++	-	데이터베이스 엔진, 서버, 터미널·클라이언트 도구
컴파일러	Visual Studio 2022 (MSVC)	17 (2022)	C++ 엔진 컴파일
빌드 도구	CMake	4.3.3	엔진 빌드
언어·도구	Rust (Cargo)	1.96.0	Tauri 백엔드
런타임	Node.js / npm	24.16.0 / 11.13.0	프론트엔드 빌드
프레임워크	Tauri	2.11.5 (CLI 2.11.2)	데스크톱 관리 도구
프레임워크	React / TypeScript / Vite	19.2.4 / 5.8.3 / 7.3.5	관리 도구 화면
라이브러리	Monaco Editor (@monaco-editor/react)	4.7.0	SQL 편집기
언어	Python	3.14.4	MCP 서버, 벤치마크 스크립트
오픈소스 라이브러리	nlohmann/json, LZ4, Catch2, SHA-1, PicoSHA2	3.11.3, 1.9.4, 2.13.10, -, -	JSON 처리, 압축, 단위 테스트, 인증 해시
비교·검증	MySQL Server / Client	8.4	호환성 및 성능 비교 검증
형상 관리	Git / GitHub	-	소스 코드 및 문서 관리

1.2 개발 내용

1.2.1 주요 개발 항목

본 프로젝트는 하나의 CMake 프로젝트에 엔진 라이브러리와 실행 파일들, 시험 파일을 두고, 그 위에 데스크톱 앱과 MCP 서버를 얹은 구조이다. 핵심은 데이터베이스 엔진 라이브러리이며, 나머지 구성요소는 이 엔진을 서로 다른 방식(터미널 · TCP · GUI · AI)으로 사용하는 클라이언트 · 서버 계층이다.

그림 1-1은 구성요소 사이의 관계를, 표 1-3은 구성요소별 역할과 기술 스택을 나타낸다.

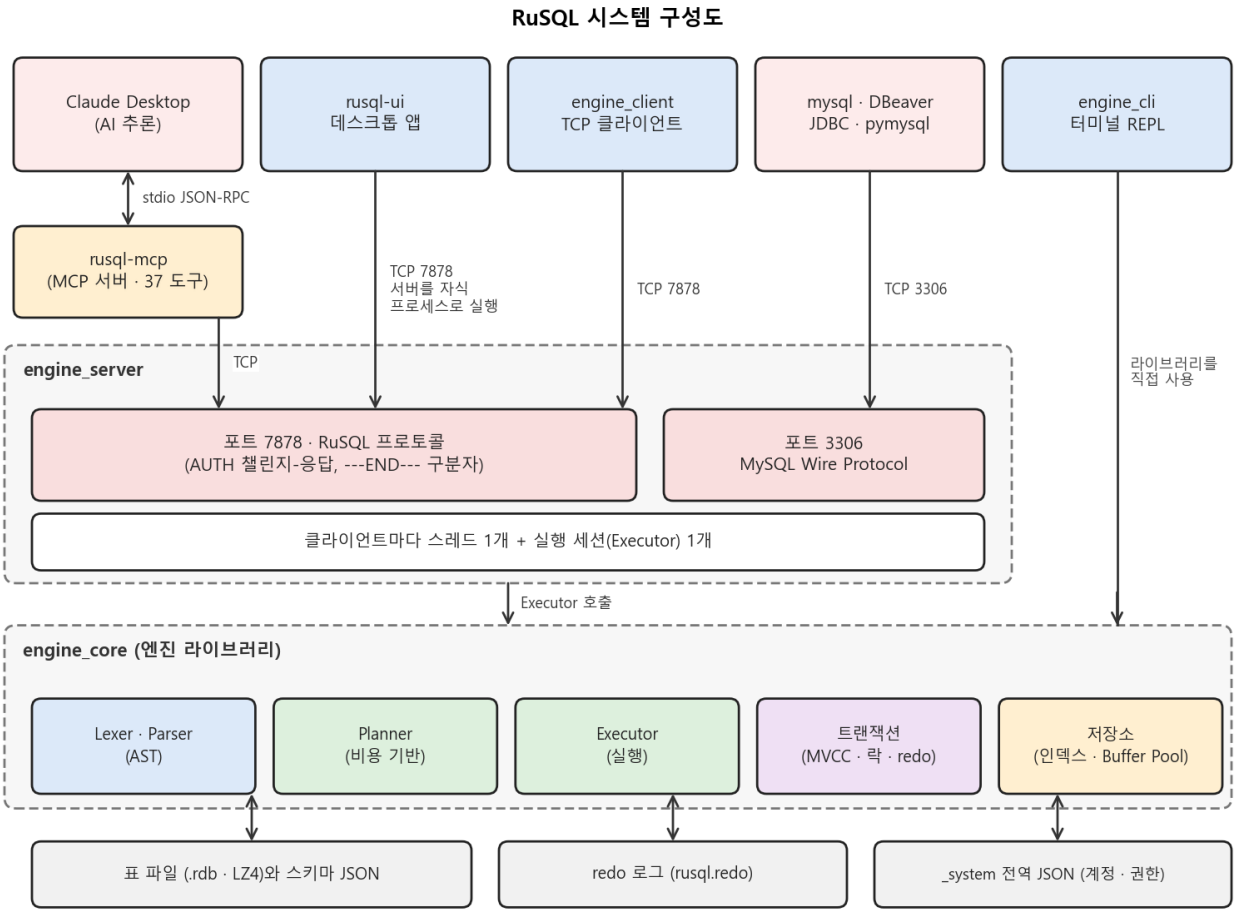


그림 1-1. RuSQL 시스템 구성도

표 1-3. 구성요소별 역할과 기술 스택

항목	역할	기술 스택
engine_core	DB 엔진 라이브러리 (Lexer · Parser · AST · 비용 기반 플래너 · Executor · B+Tree/Hash/복합 인덱스 · redo 로그 · MVCC 트랜잭션 · 행 단위 락 · Buffer Pool · 카탈로그)	C++, nlohmann/json, LZ4, picosha2, sha1
engine_cli	터미널 REPL 클라이언트 (stdin으로 SQL 직접 실행)	C++
engine_client	TCP 접속 전용 CLI 클라이언트 (-u/-p/-h/-P 옵션)	C++, 소켓
engine_server	TCP 서버 (커스텀 프로토콜 7878 + MySQL Wire Protocol 3306 동시 기동)	C++, 소켓
rusql-ui	데스크탑 GUI 클라이언트 (에디터 · 사이드바 · ERD · 서버 관리 · Diagram · B+Tree · MVCC 페이지)	Tauri(Rust), React, TypeScript, Monaco Editor
rusql-mcp	MCP 연동 — True MCP 서버 37개 도구 · Claude Desktop stdio JSON-RPC 연동 · UI 자동 연결	Python, mcp(FastMCP)

1.2.2 데이터베이스 엔진

엔진 라이브러리는 SQL 문을 Lexer, Parser, Planner, Executor의 순서로 처리한다. Lexer가 SQL 문자열을 토큰으로 나누면 Parser가 이를 문법에 맞게 AST로 만들고, 비용 기반 Planner가 통계(행 수 · 서로 다른 값 수 · MCV)를 바탕으로 접근 경로와 조인 알고리즘 · 순서 가운데 가장 비용이 낮은 계획을 고른다. 이 계획은 EXPLAIN으로 보여 주는 데 그치지 않고 실제 실행에도 그대로 쓰이며, Executor가 계획에 따라 행을 읽고 쓴다. 데이터는 표마다 LZ4로 압축한 바이너리 파일과 스키마 JSON 파일로 저장하고, 인덱스(B+Tree · Hash · 복합)는 메모리에 두었다가 엔진을 시작할 때 저장된 행에서 다시 만든다. 쓰기는 먼저 체크섬이 붙은 묶음으로 redo 로그에 기록하고 그룹 커밋으로 fsync한 뒤 응답하며, 표 파일은 체크포인트 때만 다시 쓴다. 트랜잭션은 행마다 버전을 두는 MVCC와 4단계 격리 수준으로 처리하고, 동시 접근은 행 단위 잠금(공유 · 배타 · Gap Lock · SSI의 predicate lock)과 데드락 감지로 조율한다.

그림 1-2는 SQL 문이 처리되는 순서와, 쓰기가 redo 로그를 거쳐 표 파일에 반영되고 재시작 때 복구되는 경로를 나타낸다.

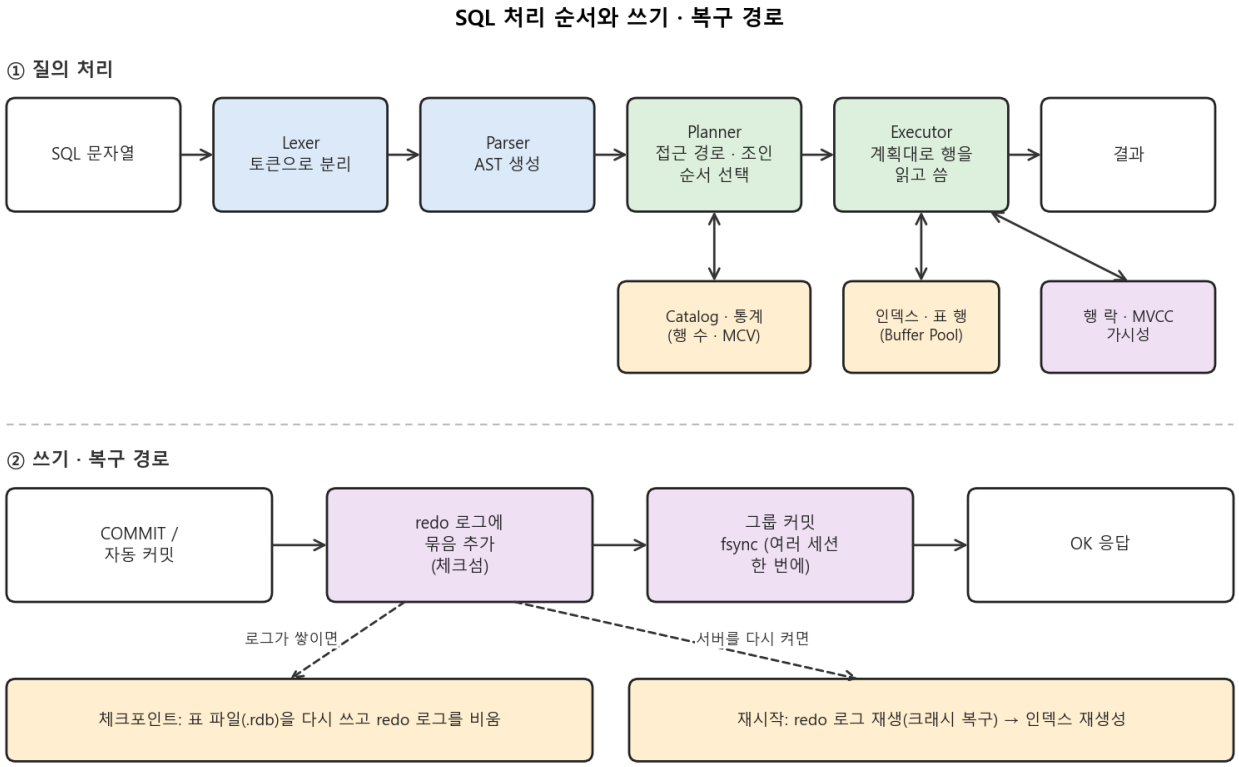


그림 1-2. SQL 처리 순서와 쓰기 · 복구 경로

1.2.3 클라이언트 · 서버 계층

서버는 두 개의 포트를 동시에 운영한다. 포트 7878은 RuSQL 고유의 텍스트 프로토콜을 쓰고, 포트 3306은 MySQL Wire Protocol을 구현해 mysql 클라이언트와 DBeaver 같은 기존 도구가 그대로 접속할 수 있다. 두 포트 모두 챌린지-응답 방식으로 인증하므로 비밀번호를 평문으로 보내지 않는다. MySQL 쪽에서는 일반 질의와 준비된 문장(prepared statement), 연결 풀이 쓰는 사용자 변경 · 연결 초기화 요청까지 처리하고, 결과 열의 타입을 함께 알려 주어 pymysql이나 JDBC 같은 드라이버가 숫자와 날짜를 제 타입으로 받는다. 실제 mysqldump 파일도 mysql 클라이언트로 그대로 적재할 수 있다. 서버는 클라이언트마다 스레드와 실행 세션을 하나씩 두고, 엔진의 공유 상태는 읽기 · 쓰기 락으로 세션 사이에 안전하게 나누어 쓴다. 터미널 클라이언트는 표준 입력으로 SQL을 받아 실행하는 REPL 프로그램이고, TCP 클라이언트는 계정과 서버 주소, 포트를 옵션으로 받아 접속한다. 데스크톱 앱은 Tauri 셸이 서버를 자식 프로세스로 띄운 뒤 TCP로 접속해 SQL을 보낸다.

그림 1-3은 두 포트와 클라이언트별 스레드, 세션이 함께 쓰는 공유 상태로 이루어진 서버 구조를 나타낸다.

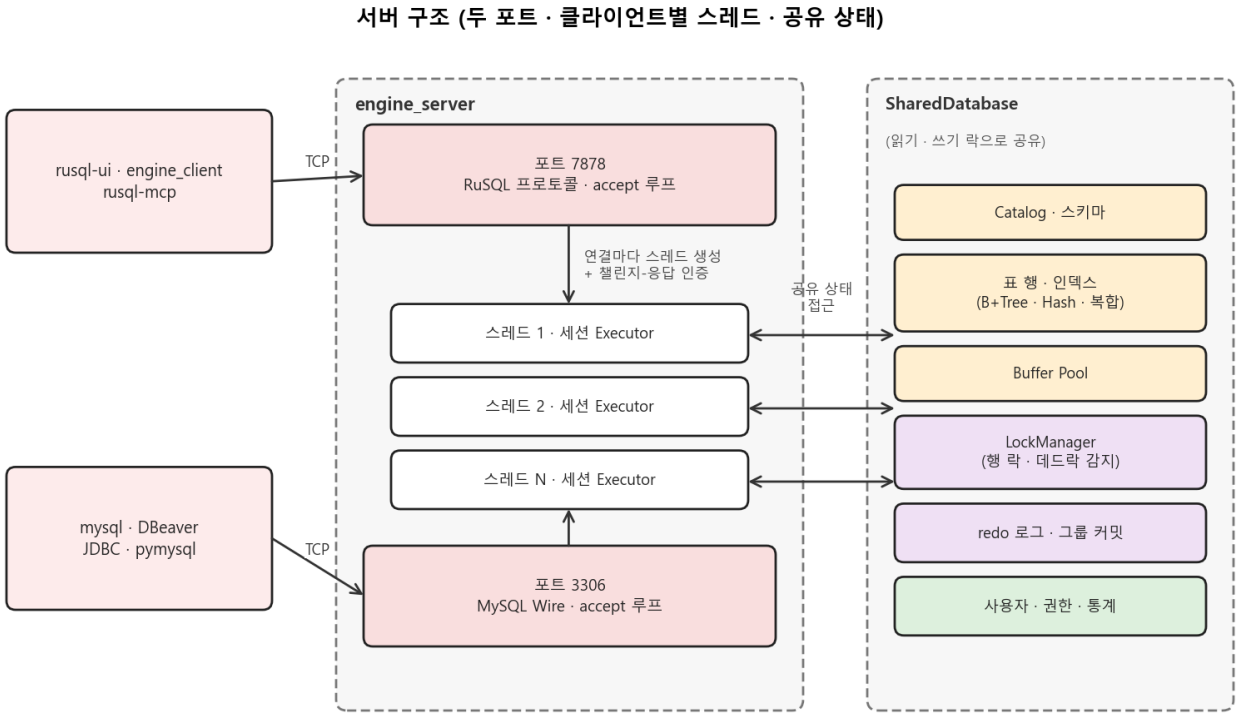


그림 1-3. 서버 구조

1.2.4 MCP 연동

MCP 서버는 FastMCP로 만든 Python 서버 하나로 구성되며, Claude Desktop과 표준 입출력(stdio) 위의 JSON-RPC로 연결된다. 노출하는 도구는 37개이고 크게 다섯 가지로 나뉜다. SQL을 실행하고 CSV 파일을 가져오거나 내보내는 도구, 데이터베이스 · 표 · 인덱스 · 락의 상태를 조회하고 진단하는 도구(스키마 확인, 실행 계획 분석, 인덱스 제안, 외래 키 관계 조회, 느린 쿼리 조회, 세션 종료, 스키마 비교 등), 서버 연결을 관리하는 도구, 앱의 쿼리 에디터를 제어하는 도구, 서버에 로그인하고 서버를 기동 · 정지 · 상태 확인하며 앱을 실행하는 서버 제어 도구이다. 위험한 SQL(DROP · TRUNCATE · WHERE 없는 UPDATE/DELETE)은 확인 없이는 거절하고, 스크립트 실행은 영향받는 행을 미리 보여 준 뒤 확인했을 때만 커밋한다. AI 추론은 Claude Desktop이 하므로 API 키가 필요 없다.

그림 1-4는 Claude Desktop에서 RuSQL 서버와 앱까지 이어지는 MCP 연동 구조를 나타낸다.

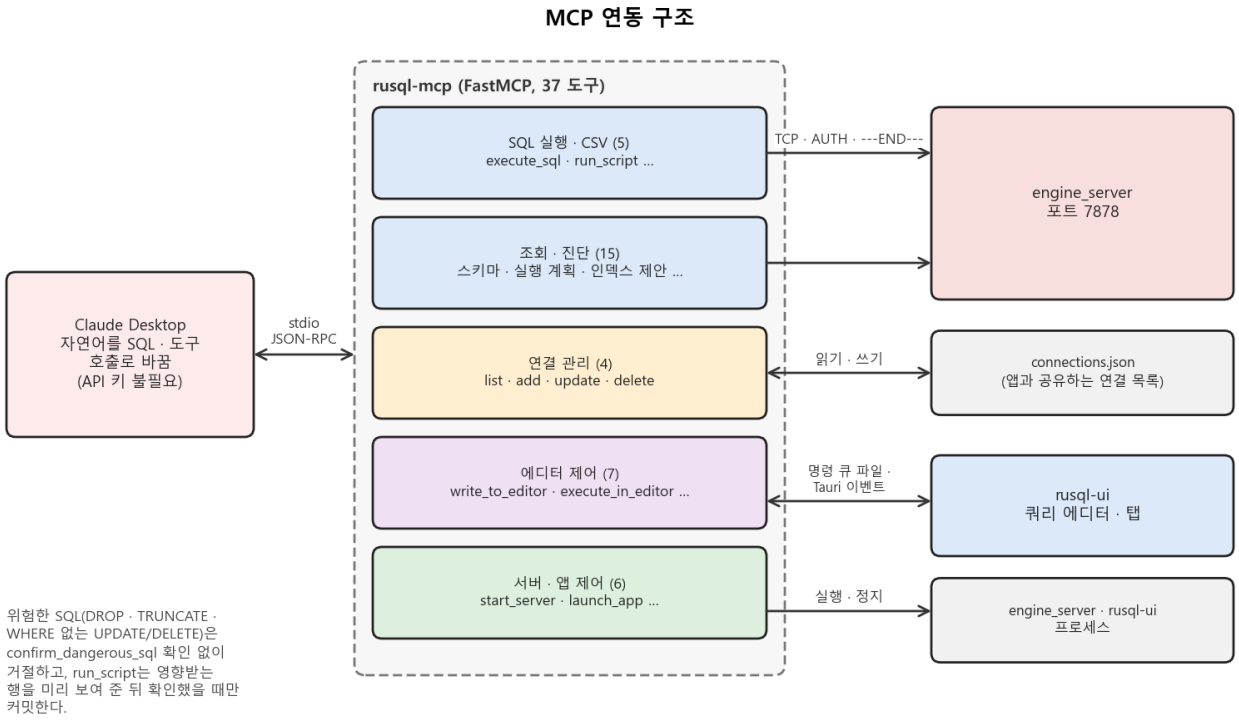


그림 1-4. MCP 연동 구조

1.2.5 개발 절차

그림 1-5는 기능을 단계별로 쌓아 가는 RuSQL의 점진적 개발 절차를 나타낸다.

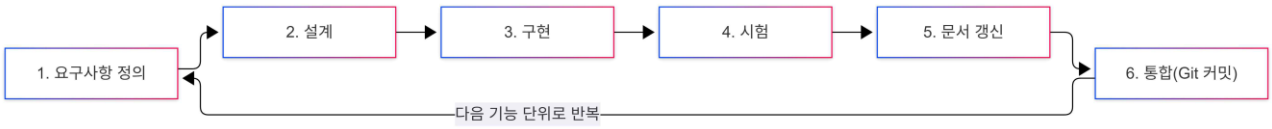


그림 1-5. RuSQL 점진적 개발 절차

1.3 용어 정의

표 1-4는 본 문서에서 사용하는 주요 용어를 정리한다.

표 1-4. 용어 정의

용어	영문	정의
RDBMS	Relational Database Management System	관계형 데이터베이스 관리 시스템 — SQL로 데이터를 관리하며 ACID를 보장하는 소프트웨어
SQL	Structured Query Language	관계형 데이터베이스를 정의 · 조작 · 제어하기 위한 표준 질의 언어
AST	Abstract Syntax Tree	SQL 문장을 파싱한 후 생성되는 추상 구문 트리 — Executor가 이를 해석하여 실행

용어	영문	정의
B+Tree	B+ Tree Index	균형 다진 트리 자료구조 — 리프 노드가 연결 리스트를 형성하여 범위 검색에 최적화된 인덱스
WAL	Write-Ahead Logging	변경 내용을 실제 데이터 파일에 쓰기 전에 로그에 먼저 기록하는 기법 — 크래시 복구 보장
Redo Log	Redo Log	COMMIT / 자동 커밋마다 바뀐 행 버전을 체크섬이 붙은 묶음으로 추가하고 fsync한 뒤 응답 — 크래시 후 재생하여 복구하며, 표 파일은 체크포인트에서만 다시 씀
MVCC	Multi-Version Concurrency Control	행에 버전 스탬프(_xmin/_xmax)를 부여해 읽기 · 쓰기 트랜잭션이 서로 간섭하지 않도록 하는 동시성 제어 기법
ACID	Atomicity/Consistency/Isolation/Durability	원자성 · 일관성 · 격리성 · 지속성 — 트랜잭션이 반드시 만족해야 하는 4가지 속성
DDL	Data Definition Language	CREATE / ALTER / DROP 등 데이터베이스 구조(스키마)를 정의하는 SQL 명령어
DML	Data Manipulation Language	INSERT / SELECT / UPDATE / DELETE 등 데이터를 조작하는 SQL 명령어
DCL	Data Control Language	GRANT / REVOKE / CREATE USER 등 접근 권한을 관리하는 SQL 명령어
MCP	Model Context Protocol (Anthropic, 2024)	AI 어시스턴트가 외부 도구에 접속하는 개방형 표준 — RuSQL은 MCP 서버로 37개 도구를 노출하여 AI 어시스턴트가 SQL을 실행하고 관리 도구의 화면을 조작
LZ4	LZ4 Compression	고속 압축 알고리즘 — .rdb 데이터 파일에 투명하게 적용하여 디스크 사용량 감소
Group Commit	Group Commit	여러 세션의 COMMIT을 하나의 fsync로 묶어 디스크 동기화 횟수를 줄이는 기법 — 처리량(TPS) 향상
Buffer Pool	Buffer Pool	디스크 페이지를 메모리에 캐싱하는 LRU 캐시 — 반복 조회 시 디스크 I/O를 줄여 응답 속도 향상 (기본 64페이지 · 16KB)
Gap Lock	Gap Lock	범위 조회 · 삽입 때 키 사이의 간격에 거는 잠금 — 같은 범위로의 동시 INSERT를 막아 팬텀 읽기를 방지 (InnoDB 방식)
SSI	Serializable Snapshot Isolation	읽은 범위(predicate)를 기록해 두었다가 팬텀이 끼어들면 COMMIT에서 실패시키는 비차단 방식의 SERIALIZABLE 구현
비용 기반 플래너	Cost-based Planner	통계(행 수 · 서로 다른 값 수 · MCV)로 접근 경로와 조인 알고리즘 · 순서의 비용을 추정해 가장 싼 계획을 고르는 구성 요소

용어	영문	정의
Collation	Collation (정렬 규칙)	글자 비교 · 정렬 · 그룹 · UNIQUE의 규칙 — 기본 utf8mb4_0900_ai_ci(대소문자 · 악센트를 구분하지 않음)로 MySQL과 같게 동작
MySQL Wire Protocol	MySQL Wire Protocol	MySQL이 사용하는 클라이언트-서버 통신 규약 — RuSQL이 이를 구현하여 DBeaver · mysql CLI · pymysql 등 기존 도구로 접속 가능
Lexer · Parser	Lexer / Parser	SQL 문자열을 토큰으로 나누고(Lexer) 문법에 맞춰 AST로 만드는(Parser) 구성 요소 — 문법 오류는 실행 전에 걸러냄
Executor	Executor (실행기)	AST로 표현된 SQL 문장을 실제로 실행하는 구성 요소 — 플래너가 고른 계획으로 행을 읽고 쓰며 트랜잭션 · 락과 함께 동작
Catalog	Catalog (카탈로그)	표 · 열 · 제약 · 파티션 같은 스키마 정보(메타데이터)를 보관하는 저장소 — Planner와 Executor가 참조
Access Path	Access Path (접근 경로)	표에서 행을 읽는 방법 — 전체를 읽는 SeqScan과 기본 키 · 보조 인덱스 · 복합 인덱스 조회 등 12종 중에서 플래너가 비용으로 선택
Join Algorithm	Join Algorithm (조인 알고리즘)	두 표의 행을 이어 붙이는 방법 — 중첩 루프 · 해시 · 정렬 병합 · 인덱스 중첩 루프 · 역방향 인덱스 중첩 루프의 5종 중에서 플래너가 선택
MCV	Most Common Values	열에서 가장 자주 나타나는 값과 그 비율 — 플래너가 조건에 맞는 행 수를 추정하는 통계로 사용 (ANALYZE TABLE로 수집)
EXPLAIN	EXPLAIN / EXPLAIN ANALYZE	쿼리의 실행 계획(접근 경로 · 조인 순서 · 예상 비용)을 보여 주는 명령 — ANALYZE는 실제로 실행해 측정한 값을 함께 표시
Hash Index	Hash Index	키의 해시 값으로 행을 찾는 인덱스 — 같다(=) 조건 조회가 O(1)에 가깝고 범위 검색에는 쓸 수 없음 (USING HASH)
Thread Pool	Thread Pool (스레드 풀)	미리 만들어 둔 스레드를 재사용해 일을 나누어 처리하는 구조 — 10k행 이상의 WHERE 필터 · GROUP BY · 정렬 · 집계를 병렬로 실행
fsync	fsync (file synchronize)	운영체제가 메모리에 모아 둔 쓰기를 디스크에 실제로 기록하도록 강제하는 호출 — COMMIT 내용이 정전에도 남도록 하며, 비용이 커서 Group Commit으로 묶음
Checkpoint	Checkpoint (체크포인트)	redo 로그에 쌓인 변경을 표 파일에 다시 써서 반영하는 시점 — 로그가 계속 커지지 않게 하고 재시작 때 재생할 양을 줄임
Undo Log	Undo Log	트랜잭션이 바꾸기 전의 값을 기록한 로그(메모리 + 디스크 영속화) — ROLLBACK과 재시작 후 미완료 변경 되돌리기에 사용
Crash Recovery	Crash Recovery (크래시 복구)	비정상 종료 뒤 재시작할 때 redo 로그를 재생해 커밋된 변경을 복원하고 미완료 변경은 되돌려 일관된 상태로 만드는 과정
TPS	Transactions Per Second	초당 처리하는 트랜잭션 수 — 데이터베이스의 처리량을 나타내는 지표

용어	영문	정의
Transaction	Transaction (트랜잭션)	하나의 작업 단위로 묶인 SQL 문들 — COMMIT으로 모두 반영하거나 ROLLBACK으로 모두 취소하며, SAVEPOINT로 중간 지점까지만 되돌릴 수 있음
Isolation Level	Transaction Isolation Level	동시에 실행되는 트랜잭션이 서로의 변경을 얼마나 볼 수 있는지 정하는 수준 — READ UNCOMMITTED · READ COMMITTED · REPEATABLE READ · SERIALIZABLE의 4단계
Phantom Read	Phantom Read (팬텀 읽기)	같은 범위 조건으로 다시 조회할 때 다른 트랜잭션이 끼워 넣은 행이 새로 나타나는 현상 — SERIALIZABLE에서 Gap Lock과 SSI로 차단
Lock	Lock (락, 잠금)	같은 데이터에 동시에 접근하는 트랜잭션을 조율하는 잠금 — 공유(읽기끼리 허용)와 배타(쓰기 단독) 락이 있고, 기다리다 시간이 지나면 오류 1205로 응답
Deadlock	Deadlock (교착 상태)	둘 이상의 트랜잭션이 서로 상대가 쥔 락을 기다리며 영원히 멈추는 상태 — 대기 관계를 DFS로 검사해 순환을 감지하고 해소
Foreign Key	Foreign Key (외래 키, FK)	한 표의 열이 다른 표의 키를 가리키도록 강제하는 제약 — 없는 부모를 가리키는 행을 막고 ON DELETE / ON UPDATE 동작(CASCADE 등)을 지정
챌린지-응답	Challenge-Response Authentication	서버가 보낸 임의의 값(챌린지)에 클라이언트가 비밀번호로 계산한 응답을 돌려보내 인증하는 방식 — 비밀번호를 평문으로 보내지 않음 (자체 프로토콜과 MySQL 프로토콜 모두 사용)
REPL	Read-Eval-Print Loop	입력한 명령을 읽어 실행하고 결과를 출력하는 일을 반복하는 대화형 방식 — 터미널 클라이언트가 표준 입력으로 SQL을 받아 실행
ERD	Entity-Relationship Diagram	표(엔티티)와 표 사이의 관계를 그림으로 나타낸 다이어그램 — RuSQL 관리 도구가 스키마와 외래 키로 그려 주고 편집도 가능
Tauri	Tauri	웹 기술로 만든 화면을 Rust 셸에 담아 만드는 데스크톱 앱 프레임워크 — RuSQL 데스크톱 앱이 사용하며 서버를 자식 프로세스로 실행
Monaco Editor	Monaco Editor	VS Code에 쓰이는 코드 편집기 엔진 — RuSQL 데스크톱 앱의 SQL 편집기에 사용
JSON-RPC	JSON-RPC (over stdio)	JSON 메시지로 원격 함수를 호출하는 규약 — Claude Desktop과 MCP 서버가 표준 입출력(stdio)으로 주고받음
FastMCP	FastMCP	Python으로 MCP 서버를 쉽게 만들 수 있게 해 주는 라이브러리 — RuSQL의 MCP 서버가 사용

2. 기능적 요구사항

요구사항은 5장의 사용사례를 기반으로 식별한 주제와 에픽, 사용자 스토리로 정의한다(프로덕트 백로그). 표 2-1은 주제와 에픽을, 표 2-2는 에픽별 사용자 스토리를 나타낸다. 2.1절부터는 각 기능 요구사항(FR)의 상세를 기술한다.

표 2-1. 주제·에픽 목록

액터	주제ID	주제명	에픽ID	에픽명	에픽설명
사용자	T-01	SQL 처리	E-01	SQL 파싱·실행	사용자가 입력한 SQL을 파싱하고 실행 계획을 세워 실행한다.
			E-02	비용 기반 질의 최적화	인덱스와 통계 정보를 이용해 비용이 가장 낮은 실행 계획을 선택한다.
사용자	T-02	트랜잭션·동시성	E-03	MVCC 트랜잭션 처리	행 단위 MVCC와 격리 수준으로 동시 트랜잭션을 처리한다.
			E-04	락·데드락 감지	락 매니저로 자원 접근을 통제하고 데드락을 감지·해소한다.
관리자	T-03	장애 복구	E-05	WAL·Undo 복구	서버 재기동 시 WAL과 Undo Log로 데이터 정합성을 복구한다.
외부 클라이언트	T-04	MySQL 프로토콜 호환	E-06	MySQL 와이어 프로토콜 서버	기존 MySQL 클라이언트가 별도 드라이버 없이 접속하도록 지원한다.
사용자	T-05	관리 도구	E-07	SQL 편집기	데스크톱 도구에서 SQL을 작성·실행하고 결과를 확인한다.
			E-08	스키마·ERD 뷰어	테이블·컬럼·관계를 조회하고 ERD로 시각화한다.
			E-09	서버 모니터링	서버 상태와 연결 정보를 조회한다.
AI 어시스턴트	T-06	MCP·AI 연동	E-10	MCP로 DB 제어	AI가 MCP 도구로 SQL을 실행하고 결과를 조회한다.
			E-11	MCP로 관리 도구 제어	AI가 MCP 도구로 관리 도구의 화면(에디터 탭 등)을 조작한다.
			E-12	위험 SQL 안전장치	AI가 DROP 등 위험 SQL을 사용자 확인 없이 실행하지 못하게 한다.

표 2-2. 사용자 스토리

에픽ID	에픽명	US_ID	사용자 스토리	추정작업량	우선순위
E-01	SQL 파싱·실행	US-01	사용자는 다양한 범위의 SQL 쿼리문을 실행하기 위하여 SQL 쿼리문을 서버에 전달한다.	8	상
		US-02	사용자는 여러 테이블을 조합한 결과를 얻기 위하여 JOIN·서브쿼리·CTE가 포함된 SQL 문을 실행한다.	8	상

에픽ID	에픽명	US_ID	사용자 스토리	추정작업량	우선순위
E-02	비용 기반 질의 최적화	US-03	사용자는 대량 데이터에서 빠른 응답을 얻기 위하여 B+Tree·Hash 인덱스를 생성한다.	5	상
		US-04	사용자는 실행 계획을 확인하기 위하여 EXPLAIN으로 질의 계획을 조회한다.	3	중
E-03	MVCC 트랜잭션 처리	US-05	사용자는 여러 세션에서 안전하게 데이터를 갱신하기 위하여 BEGIN/COMMIT/ROLLBACK으로 트랜잭션을 실행한다.	8	상
		US-06	사용자는 트랜잭션의 데이터 정합성 수준을 선택하기 위하여 4단계 격리 수준 중 하나를 지정한다.	5	중
E-04	락·데드락 감지	US-07	사용자는 동시 갱신 충돌 시 시스템이 정지하지 않도록 데드락이 자동으로 감지·해소되기를 기대한다.	5	중
E-05	WAL·Undo 복구	US-08	관리자는 서버 비정상 종료 후에도 데이터를 보존하기 위하여 재기동 시 자동 복구가 수행되기를 기대한다.	8	상
E-06	MySQL 와이어 프로토콜 서버	US-09	외부 클라이언트 사용자는 기존 도구를 그대로 쓰기 위하여 mysql CLI·DBeaver로 서버에 접속한다.	5	상
E-07	SQL 편집기	US-10	사용자는 SQL을 편리하게 작성하기 위하여 관리 도구의 편집기에서 SQL을 작성·실행한다.	5	상
E-08	스키마·ERD 뷰어	US-11	사용자는 테이블 구조를 파악하기 위하여 스키마와 ERD를 조회한다.	3	중
E-09	서버 모니터링	US-12	관리자는 서버 상태를 확인하기 위하여 연결 목록과 서버 상태를 조회한다.	2	중
E-10	MCP로 DB 제어	US-13	AI 어시스턴트는 사용자를 대신해 SQL을 실행하기 위하여 MCP execute_sql 도구를 호출한다.	5	상
E-11	MCP로 관리 도구 제어	US-14	AI 어시스턴트는 사용자의 작업을 돕기 위하여 MCP 도구로 편집기 탭을 생성·전환하고 SQL을 기입한다.	5	중
E-12	위험 SQL 안전장치	US-15	사용자는 실수로 데이터를 잃지 않기 위하여 AI가 DROP 등 위험 SQL 실행 전에 확인을 요청하기를 기대한다.	3	상

2.1 SQL 파싱 · 실행 · DDL · DML

표 2-3은 SQL 파서와 실행기, DDL, DML(기본 · 고급)에 대한 기능 요구사항(FR-001~FR-004)을 정리한다.

표 2-3. SQL 파싱 · 실행 · DDL · DML 요구사항

요구사항 ID	상세 설명	상태
FR-001:	Lexer(어휘 분석), Parser(재귀 하강 구문 분석 · Statement AST 생성), Executor(명령 실행), 세미콜	**완료**

요구사항 ID	상세 설명	상태
SQL 파서 & 실행기	론 기반 멀티 쿼리, -- · # · /* */ 주석 처리, MySQL 실행 주석(/*! … */)과 힌트(/*+ MAX_EXECUTION_TIME(n) */), 이름의 대소문자 무시 · 백틱 이름 · db.table 표기, 별칭의 AS 생략	
FR-002: DDL	CREATE/DROP TABLE(IF [NOT] EXISTS · LIKE · AS SELECT · ENGINE/CHARSET/COLLATE 등 표 옵션), ALTER TABLE(ADD/DROP/RENAME/MODIFY/CHANGE COLUMN · FIRST/AFTER · RENAME TO · ADD/DROP INDEX · UNIQUE · CHECK · FOREIGN KEY 제약 · 기본값 변경 · 여러 동작을 한 문장으로 — 전부 되거나 전부 안 됨, 이미 있는 행이 새 제약을 어기면 거절), CREATE/DROP INDEX(단일 · 복합 · UNIQUE · USING HASH), CREATE/DROP/ALTER VIEW · Updatable View(뷰에 INSERT/UPDATE/DELETE), 테이블 파티셔닝(RANGE · LIST · HASH, ADD/DROP PARTITION), DESCRIBE, SHOW CREATE TABLE · SHOW CREATE VIEW · SHOW INDEX, TRUNCATE, RENAME TABLE, 다중 데이터베이스(USE/CREATE DATABASE/DROP DATABASE/SHOW DATABASES), CREATE [OR REPLACE] SYNONYM/DROP SYNONYM [IF EXISTS]/SHOW SYNONYMS	**완료**
FR-003: DML 기본	INSERT(단일 · 다중 · SET 형식 · SELECT FROM · IGNORE · ON DUPLICATE KEY UPDATE · RETURNING), REPLACE, UPDATE(단일 · JOIN/쉼표 다중 테이블 · ORDER BY · LIMIT · RETURNING · 자기참조 식 · 같은 표를 두 번 쓰는 별칭), DELETE(단일 · 다중 테이블 · ORDER BY · LIMIT · RETURNING), SELECT(WHERE · ORDER BY(별칭 · 위치번호 · 식) · LIMIT/OFFSET/FETCH FIRST n ROWS ONLY · DISTINCT · GROUP BY(WITH ROLLUP) · HAVING · INTO @변수 · FOR UPDATE · FOR SHARE · NOWAIT · SKIP LOCKED), MERGE INTO … USING(WHEN MATCHED [AND 조건] UPDATE/DELETE · WHEN NOT MATCHED INSERT)	**완료**
FR-004: DML 고급	JOIN(INNER · LEFT · RIGHT · FULL OUTER · CROSS · NATURAL · USING · LATERAL · 쉼표 조인 · 자체 조인), 서브쿼리(WHERE/FROM/SELECT 스칼라 · 상관 서브쿼리 · EXISTS/IN/NOT IN · ANY/SOME/ALL · 행 생성자), CTE(WITH · WITH RECURSIVE), UNION[ALL] · INTERSECT[ALL] · EXCEPT[ALL], CASE WHEN · IF · CAST, 정규식(REGEXP · RLIKE), 윈도우 함수 구문(OVER · 이름 있는 WINDOW · ROWS/RANGE 프레임 · FILTER), PREPARE/EXECUTE/DEALLOCATE, 사용자 변수(@var SET/SELECT)	**완료**

2.2 함수 · 데이터 타입 · 제약조건

표 2-4는 내장 함수, 데이터 타입, 제약조건에 대한 기능 요구사항(FR-005~FR-007)을 정리한다.

표 2-4. 함수 · 데이터 타입 · 제약조건 요구사항

요구사항 ID	상세 설명	상태
FR-005: 내장 함수	집계 함수(COUNT · SUM · AVG · MIN · MAX · GROUP_CONCAT · STDDEV · VARIANCE · BIT_AND/BIT_OR · JSON_AGG · JSON_ARRAYAGG · JSON_OBJECTAGG · ANY_VALUE, DISTINCT · FILTER), 윈도우 함수 (ROW_NUMBER/RANK/DENSE_RANK/LAG/LEAD/FIRST_VALUE/LAST_VALUE/NTH_VALUE/NTILE/PERCENT_RANK/CUME_DIST), 스칼라 함수 — 문자열 (UPPER/LOWER/LENGTH/TRIM/CONCAT/SUBSTR/REPLACE/LPAD/RPAD/INSTR/FORMAT 등), 숫자(ROUND/ABS/CEIL/FLOOR/MOD/SQRT/POW/LOG/RAND/TRUNCATE 등), 날짜 · 시간	**완료**

요구사항 ID	상세 설명	상태
	(NOW/CURDATE/DATE_FORMAT/DATEDIFF/DATE_ADD/DATE_SUB/TIMESTAMPDIFF/UNIX_TIMESTAMP/LAST_DAY 등), 조건 · NULL(COALESCE/IFNULL/NULLIF), JSON(JSON_EXTRACT/UNQUOTE/VALUE · JSON_OBJECT/ARRAY · JSON_SET/INSERT/REPLACE/REMOVE · JSON_MERGE_PATCH · JSON_CONTAINS/SEARCH · JSON_KEYS/LENGTH/TYPE · MEMBER OF, ->/->> 연산자, 경로의 last · 범위 · * · **), 시스템 · 세션 (DATABASE/VERSION/USER/LAST_INSERT_ID/ROW_COUNT/FOUND_ROWS 등). 계산 결과는 진짜 MySQL 8.4.9와 같은 규칙을 따른다(DECIMAL 정확 계산과 자릿수, FLOAT 6자리, NULL 전파, 쓰는 문장의 0 나누기 오류, 글자를 수로 읽는 규칙)	
FR-006: 데이터 타입	정수(TINYINT · SMALLINT · MEDIUMINT · INT · BIGINT, UNSIGNED 범위 검사), BOOL, 실수(FLOAT · DOUBLE · REAL), DECIMAL/NUMERIC(정확한 십진수), 문자(CHAR · VARCHAR · TEXT 계열), BLOB 계열, 날짜 · 시간(DATE · TIME(n) · DATETIME(n) · TIMESTAMP(n) · YEAR, 소수 초 최대 6자리), ENUM · SET, JSON. 열 타입에 따라 값을 검사하고(범위 · 길이 · 형식이 틀리면 거절) 비교 · 정렬 · 인덱스도 열 타입을 따른다. 글자 비교 규칙은 utf8mb4_0900_ai_ci(대소문자 · 악센트 무시)	**완료**
FR-007: 제약조건 & 키	PRIMARY KEY(복합 포함) · UNIQUE(복합 포함) · NOT NULL · DEFAULT(NOW() · CURRENT_TIMESTAMP, ON UPDATE CURRENT_TIMESTAMP) · CHECK · AUTO_INCREMENT · 이름 붙은 제약(CONSTRAINT), FOREIGN KEY(ON DELETE/UPDATE CASCADE · SET NULL · SET DEFAULT · RESTRICT · NO ACTION, 표를 건너는 연쇄, 자기 자신을 가리키는 키). 동시 INSERT/UPDATE에서도 UNIQUE · 외래키 값이 지켜지도록 값 클레임으로 보호	**완료**

2.3 트랜잭션 · 스토리지 · 서버 · 프로토콜

표 2-5는 트랜잭션 · 동시성, 스토리지 · 인덱스, 서버 · 프로토콜, 서버 운영 · 진단에 대한 기능 요구사항(FR-008~FR-011)을 정리한다.

표 2-5. 트랜잭션 · 스토리지 · 서버 · 프로토콜 요구사항

요구사항 ID	상세 설명	상태
FR-008: 트랜잭션 & 동시성	Redo 로그 기반 내구성 — COMMIT · autocommit 응답 전에 체크섬이 붙은 묶음을 기록하고 그룹 커밋으로 fsync, 비정상 종료 후 재시작 시 자동 재생(같은 행의 복사본도 잃지 않음), 표 파일은 체크포인트에서만 다시 쓰기, BEGIN/START TRANSACTION [READ ONLY]/COMMIT [AND CHAIN]/ROLLBACK · SAVEPOINT/RELEASE SAVEPOINT/ROLLBACK TO SAVEPOINT · SET autocommit, 격리 수준 4단계(READ UNCOMMITTED · READ COMMITTED · REPEATABLE READ · SERIALIZABLE — 실제로 다르게 동작), 행 버전 체인 MVCC(_xmin/_xmax · 가시성 필터 · VACUUM · 자동 VACUUM), Undo Log(메모리 + 디스크 영속화), 행 단위 동시 쓰기, 블로킹 대기 락(SELECT FOR UPDATE/SHARE · 공유/배타 · 락 대기 시간 초과 오류 1205), DFS 데드락 감지, Gap Lock(팬텀 방지) · SSI predicate lock(SERIALIZABLE), LOCK TABLES/UNLOCK TABLES · SHOW LOCKS, 연결이 끊기면 열린 트랜잭션 자동 ROLLBACK, ROLLBACK은 외래키 연쇄까지 되돌림	**완료**
FR-009: 스토리지 &	B+Tree(단일 · 복합 · 커버링) · Hash Index(USING HASH, 등호 O(1)) · 보조 인덱스 증분 갱신(전체	**완료**

요구사항 ID	상세 설명	상태
인덱스	재빌드 없음) · 인덱스는 메모리에 두고 시작할 때 행에서 다시 생성, 표마다 .rdb 바이너리 저장 · LZ4 투명 압축/해제, Buffer Pool(LRU · 16KB 페이지 · 기본 64페이지 · --buffer-pool-size), 쿼리 결과 캐시(LRU 512 · 표 단위 무효화), 비용 기반 플래너(AccessPath 12종: SeqScan/PkPoint/PkBetween/PkRange/SecondaryPoint/SecondaryRange/SecondaryBetween/CompositeIndex/CompositeIndexPrefix/HashPoint/SecondaryLikePrefix/IndexIntersection, 조인 알고리즘 5종: Nested Loop/Hash/Sort-Merge/IndexNL/ReverseIndexNL 자동 선택, JOIN 순서 최적화 — System-R bitmask DP(N≤8) · 그리디 폴백, 히스토그램 · MCV 통계 ANALYZE TABLE), 병렬 쿼리(ThreadPool — WHERE 필터 · GROUP BY · ORDER BY 정렬 · 집계, 10k행 이상 자동)	
FR-010: 서버 & 프로토콜	TCP 서버(포트 7878 커스텀, ---END--- 구분자, AUTH 챌린지-응답 — 비밀번호를 평문으로 보내지 않음) · MySQL Wire Protocol(포트 3306, mysql_native_password 챌린지-응답, COM_QUERY/COM_PING/COM_INIT_DB/COM_STMT_PREPARE/EXECUTE(바이너리 프로토콜)/CLOSE/RESET/COM_CHANGE_USER/COM_RESET_CONNECTION, 결과 열 타입 전달, SHOW FULL TABLES/COLUMNS/VARIABLES/COLLATION/CHARSET/ENGINES, SELECT @@var — DBeaver · mysql CLI · JDBC · mysql-connector-python · pymysql 호환, 서버 버전 8.4.9), 실제 mysqldump 파일 적재(실행 주석 · SET 목록 · FOREIGN_KEY_CHECKS), INFORMATION_SCHEMA(tables/columns/statistics/schemata/key_column_usage/table_constraints/referential_constraints/views/processlist 등), BACKUP/RESTORE, --mysql-port/--mysql-bind/--no-mysql/--buffer-pool-size 옵션, max_connections(기본 151)	**완료**
FR-011: 서버 운영 · 진단	SHOW PROCESSLIST · KILL [QUERY | CONNECTION] n(실행 중인 SELECT · INSERT … SELECT · CALL 본문 · UPDATE/DELETE/MERGE의 행 찾기 · EXPLAIN ANALYZE · CREATE INDEX · 락 대기를 멈추거나 세션을 끊음), SHOW [GLOBAL | SESSION] STATUS(문장 종류별 수 · 연결 · 버퍼 풀 · redo · 락 대기 카운터), 느린 쿼리 로그(SET GLOBAL slow_query_log/long_query_time, information_schema.slow_log · slow_query.log), EXPLAIN · EXPLAIN ANALYZE(조인별 알고리즘 · 하위 질의 실행 수와 시간 · UPDATE/DELETE 계획) · TRACE(문장의 단계별 시간 추적), SUGGEST INDEX(SELECT · UPDATE · DELETE — 어떤 인덱스가 스캔을 멈추게 하는지 · 이유 · 읽을 행 수), ANALYZE TABLE · OPTIMIZE TABLE · CHECK TABLE · VACUUM · CHECKPOINT · SHOW BUFFER POOL/WAL, information_schema.innodb_trx/innodb_locks/innodb_lock_waits/innodb_deadlocks/row_versions/recovery, 크래시 복구 기록(걸린 시간 · 되살린 변경 수), 실행 시간 제한(MAX_EXECUTION_TIME)	**완료**

2.3.1 격리 수준과 락 동작

표 2-6은 격리 수준별로 Dirty Read, Non-repeatable Read, Phantom Read가 나타나는지를, 표 2-7은 락의 종류별로 서로 기다리는지를 정리한다. 두 표는 엔진을 직접 실행해 확인한 동작이며 표 2-5의 FR-008을 보충한다.

표 2-6. 격리 수준별 읽기 이상 현상

격리 수준	Dirty Read	Non-repeatable Read	Phantom Read	읽기 방식
READ UNCOMMITTED	발생	발생	발생	다른 트랜잭션이 아직 커밋하지 않은 변경도 읽는다.
READ COMMITTED (기본값)	방지	발생	발생	문장마다 그 시점에 커밋된 최신 값을 읽는다.
REPEATABLE READ	방지	방지	방지	트랜잭션이 처음 읽은 시점의 스냅샷을 계속 읽는다. 범위를 FOR UPDATE · FOR SHARE로 읽으면 Gap Lock으로 그 범위의 INSERT를 기다리게 한다.
SERIALIZABLE	방지	방지	방지	REPEATABLE READ와 같은 스냅샷 읽기에 더해, 읽은 행이 바뀌었거나 읽은 범위에 새 행이 끼어들었으면 COMMIT에서 직렬화 오류로 실패시킨다(SSI). 일반 SELECT는 기다리게 하지 않는다.

기본 격리 수준은 READ COMMITTED이다(MySQL의 기본값은 REPEATABLE READ). SET [SESSION] TRANSACTION ISOLATION LEVEL로 바꾸고 SHOW ISOLATION LEVEL로 확인한다.

표 2-7은 행이 이미 잡혀 있는 락, 열이 다른 트랜잭션이 요청하는 동작을 나타낸다. '대기'는 락이 풀릴 때까지 기다린다는 뜻이고, '—'는 서로 다른 대상이라 충돌을 따지지 않는다는 뜻이다.

표 2-7. 락 호환성

보유 중인 락	공유 락(S) 요청	배타 락(X) 요청	일반 SELECT	Gap Lock 요청	범위 안 INSERT
공유 락(S) — FOR SHARE	허용	대기	허용	—	—
배타 락(X) — UPDATE · DELETE · FOR UPDATE	대기	대기	허용	—	—
Gap Lock — 범위 FOR UPDATE · FOR SHARE	—	—	허용	허용	대기

일반 SELECT는 MVCC 스냅샷을 읽으므로 락을 잡지도 기다리지도 않고, 같은 표의 서로 다른 행을 고치는 문장끼리도 기다리지 않는다.

Gap Lock은 REPEATABLE READ와 SERIALIZABLE에서, 기본 키가 하나인 표의 범위 조건을 SELECT … FOR UPDATE · FOR SHARE로 읽을 때 건다. SERIALIZABLE의 일반 SELECT는 읽은 범위를 기록해 두었다가 COMMIT에서 확인한다(predicate lock).

행 락은 COMMIT이나 ROLLBACK에서 풀린다. 락을 @lock_wait_timeout(기본 50초) 안에 얻지 못하면 오류 1205로 응답하고, 데드락이 생기면 순환을 완성한 트랜잭션이 오류 1213으로 중단된다.

LOCK TABLES는 이를 쓰는 세션끼리만 서로 막고(이미 잠긴 표를 잠그면 기다리지 않고 바로 오류), 쓰지 않는 세션은 막지 않는다.

2.4 클라이언트 · UI · AI

표 2-8은 클라이언트 · UI와 MCP 연동에 대한 기능 요구사항(FR-012~FR-013)을 정리한다.

표 2-8. 클라이언트 · UI · AI 요구사항

요구사항 ID	상세 설명	상태
FR-012: 클라이언트 & UI	터미널 클라이언트(REPL, stdin 직접 실행) TCP 클라이언트(-u/-p/-h/-P 인증, 멀티라인 SQL) 데스크톱 앱(Tauri + React + Monaco Editor): 홈 화면(퀵 액션 · 연결 카드 · 연결 검색 · 즐겨찾기 · 최근 연결 · RDBMS 소개 · 액티비티 바) 에디터(Monaco · 스키마를 아는 자동완성 · 분할 에디터 · 탭 시스템(탭별 결과 보존 · 우클릭 메뉴 · 고정 · 이름 변경) · SQL 포매터(Ctrl+Shift+F) · 쿼리 히스토리 200개 · 북마크) 사이드바(DB 트리 · 컬럼 상세 PK/FK 뱃지 · 트리거 · FK 섹션 · Edit Table 모달 · 테이블 검색 · 우클릭 메뉴) 결과 영역(Canvas 자동 너비 · 셀 직접 편집 · 헤더 정렬 · 실시간 검색 · 100행 페이징 · CSV 가져오기/내보내기 · JSON 내보내기 · 막대/꺾은선/파이 차트) EXPLAIN 계획 그림(상자와 화살표, 가장 느린 단계 표시) · EXPLAIN ANALYZE 단계 막대 · 인덱스 제안 카드(임시 인덱스로 전 · 후 시간 측정) · 문장 Journey(문장이 지나는 단계 재생) · 스키마 비교(Compare Schema)와 마이그레이션 SQL ERD Editor(외래키 편집기와 DDL 미리보기, 열을 끌어서 연결, 팬/줌) Server Manager(연결 구성 폼 · CLI/MySQL/Bench/Session/Metrics/AI MCP 슬라이드 패널 · 세션 · 트랜잭션 · 락 실시간 패널과 KILL · 크래시 복구 카드 · 느린 쿼리 로그 · 추세 그래프) Diagram · B+Tree · MVCC 설명 페이지 data/_system/ 전역 파일 격리 · 연결별 독립 디렉터리(data/local/, data/data_N/)	**완료**
FR-013: MCP 연동 (Claude Desktop)	True MCP 서버(FastMCP, stdio JSON-RPC) 37개 도구: SQL 실행 · CSV 입출력(execute_sql · confirm_dangerous_sql · run_script · import_csv · export_csv) 조회 · 진단(list_databases · list_tables · get_table_schema · explain_query · trace_query · suggest_indexes · get_indexes · get_table_stats · get_fk_graph · list_locks · health_report · sample_data · slow_queries · kill_session · diff_schema) 연결 관리(list_connections · add_connection · update_connection · delete_connection) 앱 에디터 제어(write_to_editor · new_editor_tab · close_editor_tab · switch_editor_tab · list_editor_tabs · get_editor_tab_content · execute_in_editor) 서버 · 앱 제어(login · start_server · stop_server · get_server_status · launch_app · list_app_instances). 요청마다 RuSQL 포트 7878 TCP 연결 · AUTH 인증 · ---END--- 구분자 수신, SELECT 결과는 구조화된 JSON 배열. 위험한 SQL(DROP · TRUNCATE · WHERE 없는 UPDATE/DELETE)은 confirm_dangerous_sql 확인 없이는 거절, run_script는 영향받는 행을 미리 보여 주고 commit=True일 때만 저장. AI 추론은 Claude Desktop(클라이언트 측)이 담당하므로 API 키 불필요. UI 자동 연결: Server Manager AI MCP 패널의 'Claude Desktop 자동 연결' 버튼 →	**완료**

요구사항 ID	상세 설명	상태
	setup_mcp_config Tauri 커맨드가 mcp 패키지가 설치된 Python을 찾아 claude_desktop_config.json을 일반 설치 · Windows Store 두 경로에 BOM 없는 UTF-8로 생성 · 병합	

2.5 DCL · 사용자 관리 · 역할 · 동의어

표 2-9는 DCL · 사용자 관리 · 역할 · 동의어와 저장 프로시저 · 트리거 · UDF · 백업에 대한 기능 요구사항(FR-014~FR-015)을 정리한다.

표 2-9. DCL · 사용자 관리 · 역할 · 동의어 요구사항

요구사항 ID	상세 설명	상태
FR-014: DCL · 사용자 관리 · 역할 · 동의어	사용자 관리: CREATE USER [IF NOT EXISTS] 'user'@'host' [IDENTIFIED BY 'password'], DROP USER [IF EXISTS], 비밀번호는 SHA-256 해시와 mysql_native_password 해시(챌린지-응답)로 저장, 기본 root 계정 보장. 권한 제어: GRANT privilege[,...] ON db.tbl TO 'user'@'host' [WITH GRANT OPTION], REVOKE, SHOW GRANTS [FOR 'user'@'host'], SELECT/INSERT/UPDATE/DELETE/CREATE/DROP/ALTER/INDEX/ALL 권한을 DB별 · 사용자별로 분리. 역할(ROLE): CREATE ROLE, DROP ROLE [IF EXISTS], GRANT ROLE r TO 'user'@'host' [WITH ADMIN OPTION], REVOKE ROLE, GRANT 권한 ON … TO 역할, SHOW ROLES. 동의어(SYNONYM): CREATE [OR REPLACE] SYNONYM, DROP SYNONYM, SHOW SYNONYMS, SELECT/INSERT/UPDATE/DELETE에서 실제 테이블처럼 사용. 영속화(_users.json, _grants.json, _roles.json, _role_grants.json, _synonyms.json)	**완료**
FR-015: 저장 프로시저 · 트리거 · UDF · 백업	저장 프로시저: CREATE PROCEDURE/DROP PROCEDURE, CALL procedure(args), BEGIN…END 본문, IN/OUT/INOUT 매개변수, DECLARE · SET 변수(선언한 타입을 따름), IF/ELSEIF/ELSE, WHILE/LOOP/REPEAT/UNTIL, LEAVE/ITERATE · 라벨, 본문의 DML · SELECT · SELECT … INTO · CALL, 영속화(_procedures.json). 트리거: CREATE TRIGGER/DROP TRIGGER, BEFORE/AFTER INSERT/UPDATE/DELETE FOR EACH ROW, 행마다 실행되며 NEW/OLD 행 값을 읽음(BEFORE INSERT · BEFORE UPDATE에서는 SET NEW.col로 값 수정), 영속화(_triggers.json). 사용자 정의 함수(UDF): CREATE FUNCTION name(params) RETURNS type RETURN expr, DROP FUNCTION, SELECT/UPDATE에서 일반 함수처럼 호출, 영속화(_functions.json). 백업 · 복원: BACKUP [DATABASE db] [INTO 'file'] — mysqldump 스타일 SQL 덤프(DROP TABLE IF EXISTS + CREATE TABLE + INSERT, FK 동작 포함), RESTORE FROM 'file' [DATABASE db].	**완료**

3. 비기능적 요구사항

비기능적 요구사항(NFR)은 성능, 신뢰성, 보안, 사용성, 이식성, 호환성의 여섯 관점으로 나누어 정의한다. 표의 카테고리 열은 각 요구사항이 속한 세부 분류를 나타낸다.

3.1 성능 요구사항

표 3-1은 인덱스 조회와 질의 계획, 처리 시간에 관한 성능 요구사항(NFR-001~NFR-003)을 정리한다.

표 3-1. 성능 요구사항

카테고리	요구사항 ID	설명	상태
성능	NFR-001	B+Tree(O(log N)) · Hash(O(1)) 인덱스 조회, 인덱스를 타는 UPDATE/DELETE · 집계 · ORDER BY … LIMIT, 단순 쿼리 응답은 로컬에서 밀리초 단위	**완료**
성능	NFR-002	비용 기반 플래너가 조인 알고리즘 5종과 JOIN 순서를 자동 선택, 10k행 이상 SeqScan · GROUP BY · 정렬을 ThreadPool로 병렬 처리	**완료**
성능	NFR-003	행 수가 늘면 시간이 제곱으로 늘던 구간(LEFT/RIGHT/FULL JOIN · 상관 서브쿼리 · WHERE 서브쿼리 · MERGE)을 해시로 선형화 — 예: MERGE 대상 300행 × 원본 30만 행 109초 → 0.85초, COMMIT 비용 8배 절감, 대량 INSERT 2.3배 · 스캔 3~11배 · 집계 · 조인 3~8배 개선	**완료**

3.2 신뢰성 요구사항

표 3-2는 장애 복구, 롤백, 검증 체계, 동시성 제어, 참조 무결성에 관한 신뢰성 요구사항(NFR-004~NFR-009)을 정리한다.

표 3-2. 신뢰성 요구사항

카테고리	요구사항 ID	설명	상태
신뢰성	NFR-004	Redo 로그 + 그룹 커밋 fsync + 크래시 복구 — 비정상 종료 후 재시작하면 커밋된 변경을 자동 재생하여 ACID와 무결성을 복구, 복구 시간 · 되살린 변경 수는 information_schema.recovery로 확인	**완료**
신뢰성	NFR-005	ROLLBACK 후 인덱스 · 외래키 연쇄를 포함해 완전 원상 복구 (SAVEPOINT 중간 복원 포함), 표 파일에는 커밋된 행만 기록하므로 ROLLBACK 뒤 크래시해도 되돌린 변경이 되살아나지 않음	**완료**
신뢰성	NFR-006	검증 체계 — 전체 시험 946 케이스 / 1,520,540 assertions(Debug · Release, DML 인덱스 강제 설정 포함), 빌드 간 차분(같은 무작위 SQL 4만 개 이상의 출력을 글자 단위로 비교), 크래시 퍼저(kill -9 후 복구 상태 비교), 무작위 차분 퍼저, 일부러 버그를 심어 시험이 잡는지 보는 mutation 시험	**완료**
동시성	NFR-007	행 단위 락 + MVCC + 블로킹 대기 락 + DFS 데드락 감지 — 같은 표의 서로 다른 행을 여러 클라이언트가 동시에 쓰고, 락 대기 시간 초과는 MySQL 오류 1205로 응답	**완료**

카테고리	요구사항 ID	설명	상태
동시성	NFR-008	SERIALIZABLE 격리 수준에서 Gap Lock과 SSI predicate lock으로 팬텀 읽기를 차단, 그룹 커밋으로 fsync 최소화	**완료**
제약	NFR-009	외래 키(FK) 참조 무결성 100% 보장 — 잘못된 참조 INSERT/DELETE 차단, 동시 실행에서도 값 클레임으로 고아 행이 생기지 않음	**완료**

3.3 보안 요구사항

표 3-3은 인증, 권한 제어, AI 연동 시 위험 SQL 보호에 관한 보안 요구사항(NFR-010~NFR-012)을 정리한다.

표 3-3. 보안 요구사항

카테고리	요구사항 ID	설명	상태
보안	NFR-010	사용자 계정 기반 로그인 인증 — 챌린지-응답(비밀번호를 평문으로 보내지 않음), SHA-256 · mysql_native_password 해시로 저장, 기본 root 계정 보장	**완료**
보안	NFR-011	GRANT / REVOKE 권한 제어 (WITH GRANT OPTION) — DB별 · 사용자별 권한 분리, ROLE 기반 권한 그룹 관리 (WITH ADMIN OPTION)	**완료**
보안	NFR-012	MCP 위험 SQL 보호 — DROP · TRUNCATE · WHERE 없는 UPDATE/DELETE는 confirm_dangerous_sql 확인 없이는 거절, run_script는 영향받는 행을 미리 보여 준 뒤 확인했을 때만 커밋	**완료**

3.4 사용성 요구사항

표 3-4는 화면 · 명령줄 · MCP로 사용하는 방법에 관한 사용성 요구사항(NFR-013)을 정리한다.

표 3-4. 사용성 요구사항

카테고리	요구사항 ID	설명	상태
사용성	NFR-013	GUI(Tauri · React · Monaco) · CLI · TCP 클라이언트 · MySQL 클라이언트로 SQL을 입력 · 실행 · 확인, 오류는 MySQL 오류 번호로 응답, AI MCP 패널에서 버튼 한 번으로 Claude Desktop 자동 연결(비기술 사용자도 자연어로 조회 가능)	**완료**

3.5 이식성 요구사항

표 3-5는 빌드 구성과 플랫폼 독립성, 유지보수에 관한 이식성 요구사항(NFR-014~NFR-015)을 정리한다.

표 3-5. 이식성 요구사항

카테고리	요구사항 ID	설명	상태
이식성	NFR-014	CMake 기반 C++ 단일 코드베이스(Windows에서 개발 · 검증, 소켓 계층만 플랫폼 분기 — 다른 OS 빌드는 검증하지 않음), MySQL Wire Protocol로 기존 클라이언트 도구 재사용 가능, 데이터는 .rdb + JSON 파일로 플랫폼에 묶이지 않음	**완료**
유지보수	NFR-015	CMake 프로젝트 하나에 정적 라이브러리와 실행 파일들, 시험 파일을 두고 엔진 소스는 기능별 파일로 분리	**완료**

3.6 호환성 요구사항

표 3-6은 MySQL 프로토콜과 질의 결과 호환성에 관한 요구사항(NFR-016~NFR-017)을 정리한다.

표 3-6. 호환성 요구사항

카테고리	요구사항 ID	설명	상태
호환성	NFR-016	MySQL Wire Protocol 지원 — DBeaver · mysql CLI · JDBC · pymysql 등 외부 도구 직접 접속, 서버 버전 8.4.9로 응답, 실제 mysqldump 파일 적재	**완료**
호환성	NFR-017	진짜 MySQL 8.4.9에 같은 문장을 보내 결과를 비교하는 호환성 점수판 — 코퍼스 49개, 문장 4,209개에서 차이 0, 기본 정렬 규칙 utf8mb4_0900_ai_ci로 글자 비교 · 정렬 · UNIQUE도 MySQL과 같음	**완료**

4. 사용자 인터페이스 (UI) 명세

User Interface Specification

사용자 인터페이스는 데스크톱 관리 도구와 터미널 클라이언트로 구성한다. 4.1절은 화면별 구성을, 4.2절은 기능별 설계를, 4.3절은 실제 화면을 나타낸다.

4.1 화면 설계

표 4-1은 화면별 구성 레이아웃과 핵심 특징을 정리하며, 각 화면의 실제 모습은 4.3절에 나타낸다.

표 4-1. 화면별 구성과 핵심 특징

화면명	구성 레이아웃	핵심 특징
홈 화면	퀵 액션 버튼 3종 / RDBMS 소개 / 연결 카드 그리드 / 연결 검색 / 상태 표시줄 / ActivityBar	저장된 연결을 카드로 보여 주고 즐겨찾기(★) · 최근 연결 · 검색으로 빠르게 접속
메인 에디터 뷰	ActivityBar(좌측 아이콘) / 사이드바(DB 트리) / Monaco Editor / 결과 테이블 · 계획 그림 / 드래그 구분선	SQL 작성 · 실행 · 결과 확인이 한 화면에, 구분선 드래그로 레이아웃 조절, 좌우 분할 에디터(독립 Monaco 인스턴스), 스키마를 아는 자동완성
ERD Editor 뷰	표 카드 캔버스 / FK 관계선 / 하단 데이터 패널 / 외래키 편집기	팬 · 줌, 카드 드래그, 열을 끌어 다른 열에 놓으면 외래키 DDL 미리보기 후 적용
서버 관리 뷰	연결 구성 폼(이름 · 포트 · 사용자) / 시작 · 중지 · 저장 / 활동 로그 / 우측 패널 바(CLI · MySQL · Bench · Session · Metrics · AI MCP)	TCP 서버(7878) + MySQL 프로토콜(3306) 동시 기동 · 제어, 세션 · 트랜잭션 · 락 실시간 패널, 크래시 복구 카드
Server Manager AI MCP 패널	'Claude Desktop 자동 연결' 버튼 / 연결 상태 안내 텍스트 / 슬라이드 패널 토글	MCP 설정 자동 구성 — Claude Desktop 설정 파일을 두 경로에 자동 생성 · 병합, mcp 패키지 Python 자동 탐지, API 키 불필요
Diagram · B+Tree · MVCC 페이지	시스템 전체 구조 그림 / B+Tree 노드 분할 장면 / MVCC 버전 장면	구조 그림은 한국어 · EN 전환, B+Tree와 MVCC 장면은 조작 없이 스스로 재생
engine_cli	터미널 REPL — 입력 프롬프트 / 실행 결과 / 실행 시간 표시	stdin으로 SQL을 입력하면 즉시 결과 확인, 파일 리다이렉트로 SQL 스크립트 실행

4.2 기능 설계

표 4-2는 기능 영역별 주요 기능을 정리한다.

표 4-2. 기능 영역별 주요 기능

기능 영역	주요 기능
SQL 편집	Monaco Editor 문법 강조 / 스키마를 아는 자동완성(표 · 열 · 함수 · 키워드, 별칭 해석) / Ctrl+Enter 실행(선택 영역이 있으면 선택 영역만) / 세미콜론(;) 기준 멀티 쿼리 · BEGIN…END 블록 인식 분리 / SQL 포매터(Ctrl+Shift+F) / Ctrl+T(새 탭) · Ctrl+W(닫기) / 탭 고정 · 이름 변경 · 드래그 · 우클릭 컨텍스트 메뉴 / 다중 탭

기능 영역	주요 기능
	(탭별 쿼리 자동 저장 · 복원) / 주석(--, #, /* */) 지원
사이드바	DB · 테이블 · 뷰 · 인덱스 · 트리거 계층 트리 / 컬럼 상세(타입 배지 · PK · FK · NOT NULL · UNIQUE 뱃지) / DB명 더블클릭 USE 전환 / 테이블 우클릭 메뉴(Select Rows · Describe · Show Create · Copy as INSERT · Truncate · DROP · Import CSV) / 사이드바 테이블 검색 / 드래그로 사이드바 너비 조절 / 두 데이터베이스의 스키마 비교와 마이그레이션 SQL
결과 영역	컬럼 헤더 + 행 데이터 / 페이지네이션(100행) / 헤더 클릭 정렬 / 실시간 검색 / 셀 직접 편집(단일 테이블 UPDATE 자동 생성) / CSV · JSON 내보내기 / CSV 가져오기 / Canvas measureText 컬럼 자동 너비(CJK 포함) / 막대 · 꺾은선 · 파이 차트 / 쿼리 히스토리(최대 200개)
실행 계획 · 인덱스 제안	EXPLAIN의 계획을 상자와 화살표로 그린 그림(가장 느린 단계 표시) / EXPLAIN ANALYZE의 단계별 시간 막대 / 계획 아래 인덱스 제안 카드(SUGGEST INDEX — 에디터에 넣기 · 복사 · 임시 인덱스로 전 · 후 시간 재 보기) / 문장 하나가 지나는 단계를 재생하는 Journey
ERD	FK 관계선 / 카드 드래그 / 팬 · 줌 / 하단 데이터 패널 / 외래키 편집기(DDL 미리보기와 함께 편집) / 열을 끌어서 다른 열에 연결
서버 관리	TCP 서버 시작/중지(포트 7878 · 3306 동시 기동) / 활동 로그 / 세션 · 트랜잭션 · 락 실시간 패널(KILL, 다른 트랜잭션을 막는 세션과 오래 도는 문장 강조) / 느린 쿼리 로그 켜기와 목록 / 크래시 복구 카드(복구 시간 · 되살린 변경 수) / Metrics 패널(초당 문장 수 · 세션 · 락 대기 · redo 크기 · 버퍼 풀 추세) / Bench 패널
MCP 연동	Server Manager AI MCP 패널 — Claude Desktop 자동 연결(설정 파일을 두 경로에 생성 · 병합) / True MCP 37개 도구로 Claude Desktop이 RuSQL을 직접 질의하고 앱의 쿼리 에디터까지 제어 / 비기술 사용자도 자연어로 데이터 조회 · 조작 가능, API 키 불필요
단축키	Ctrl+Enter: 전체/선택 영역 실행 / Ctrl+T: 새 탭 / Ctrl+W: 탭 닫기 / Ctrl+Shift+F: SQL 포맷

4.3 화면 배치도

각 화면의 실제 배치를 보여 주는 그림이다. 그림 4-1부터 그림 4-10까지는 실행 중인 앱을 캡처한 화면이고, 그림 4-11은 터미널 클라이언트의 실제 출력 화면이다.

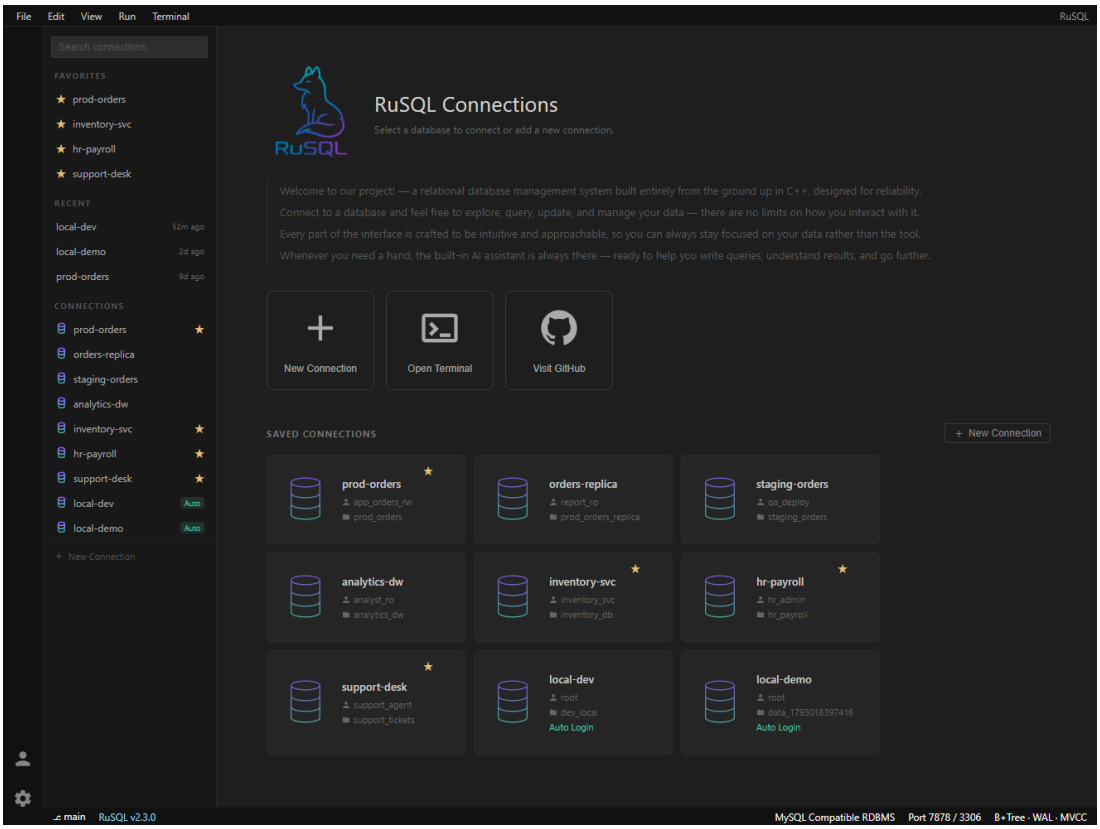


그림 4-1. 홈 화면

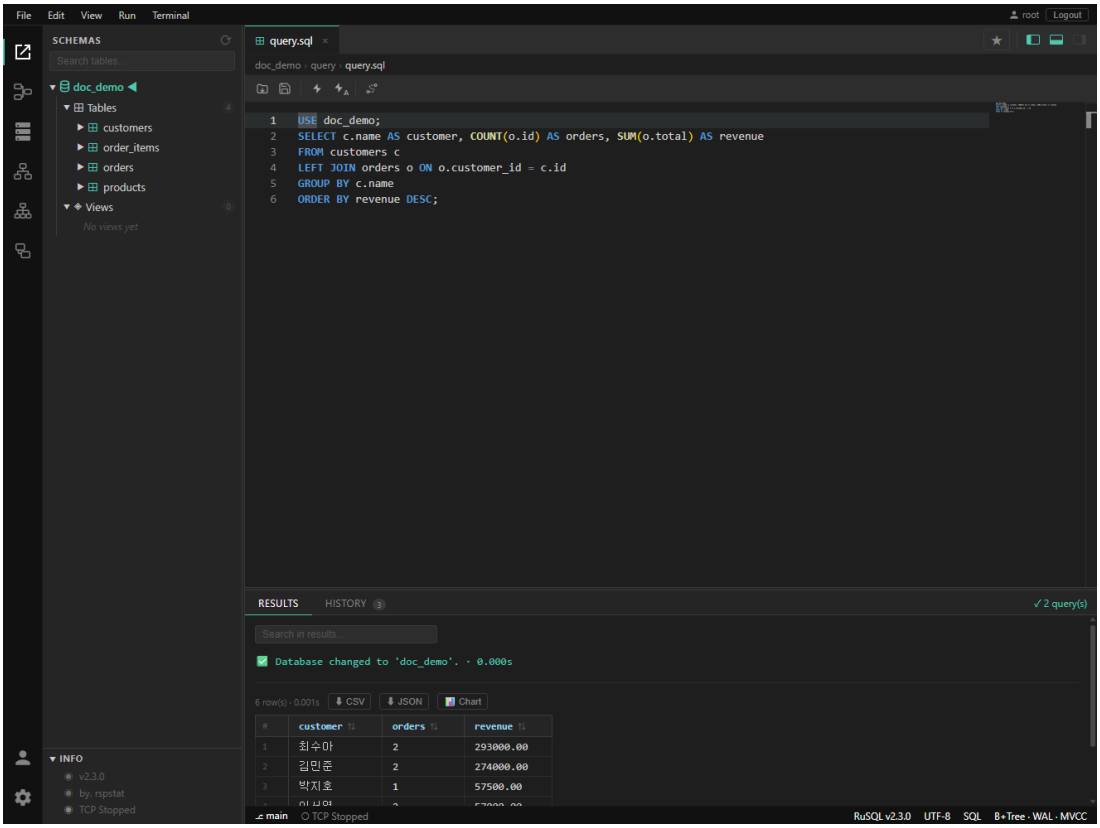


그림 4-2. 메인 에디터 뷰 (사이드바 · 에디터 · 결과)

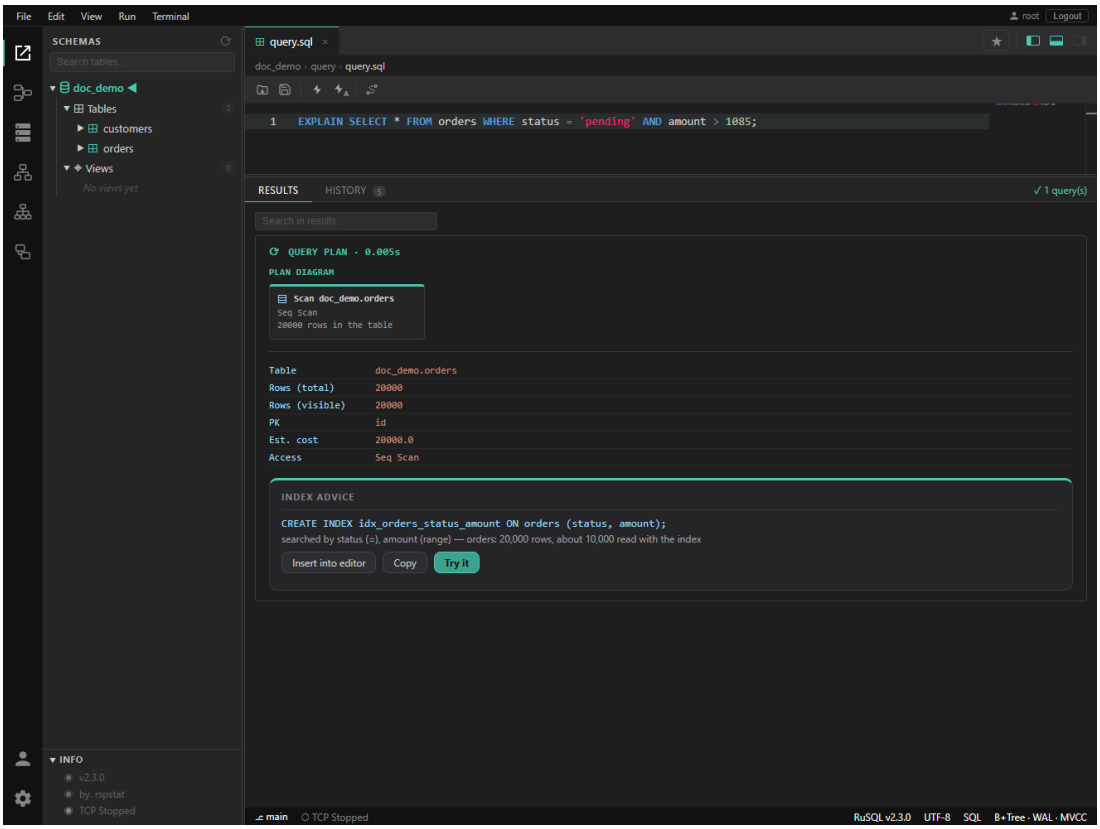


그림 4-3. 실행 계획과 인덱스 제안

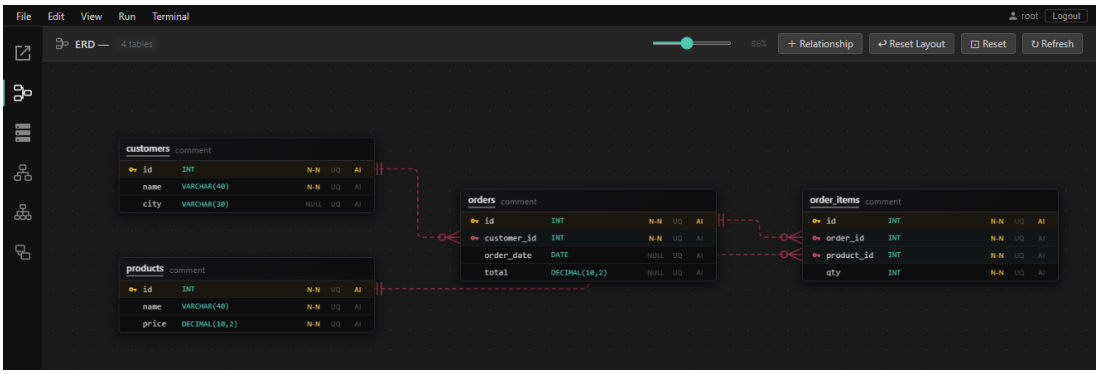


그림 4-4. ERD Editor 뷰

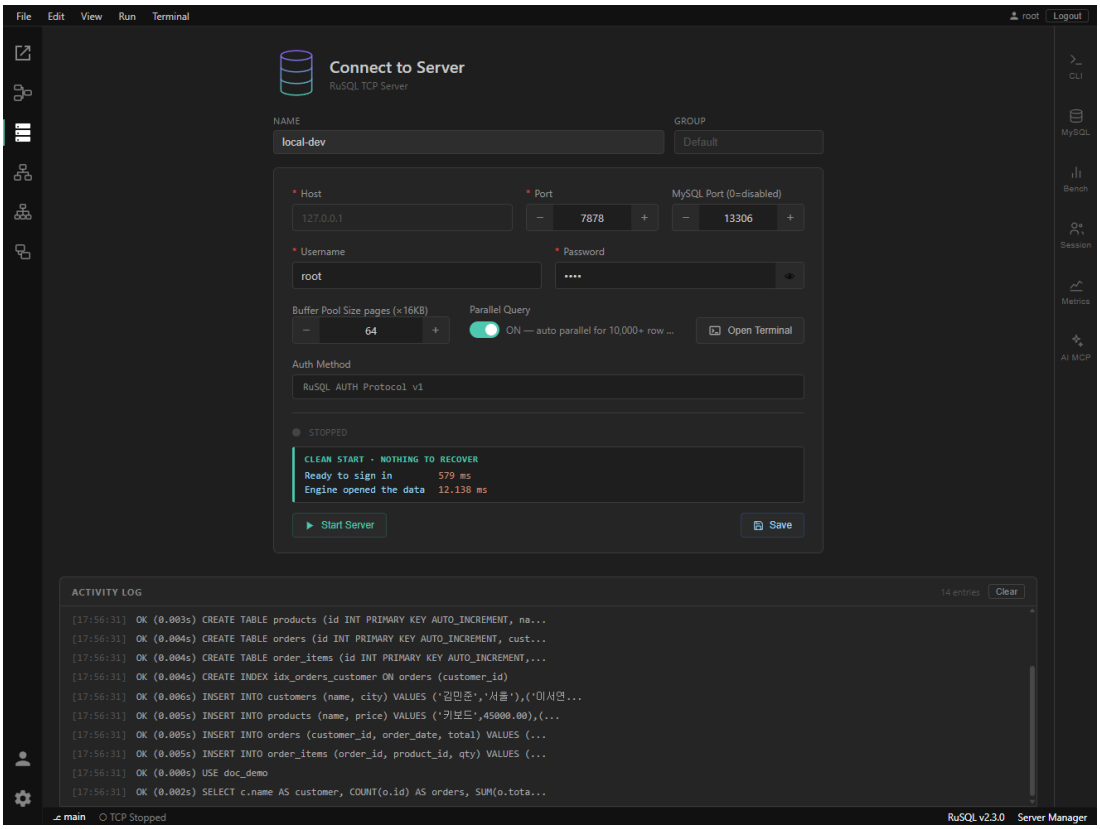


그림 4-5. 서버 관리 뷰

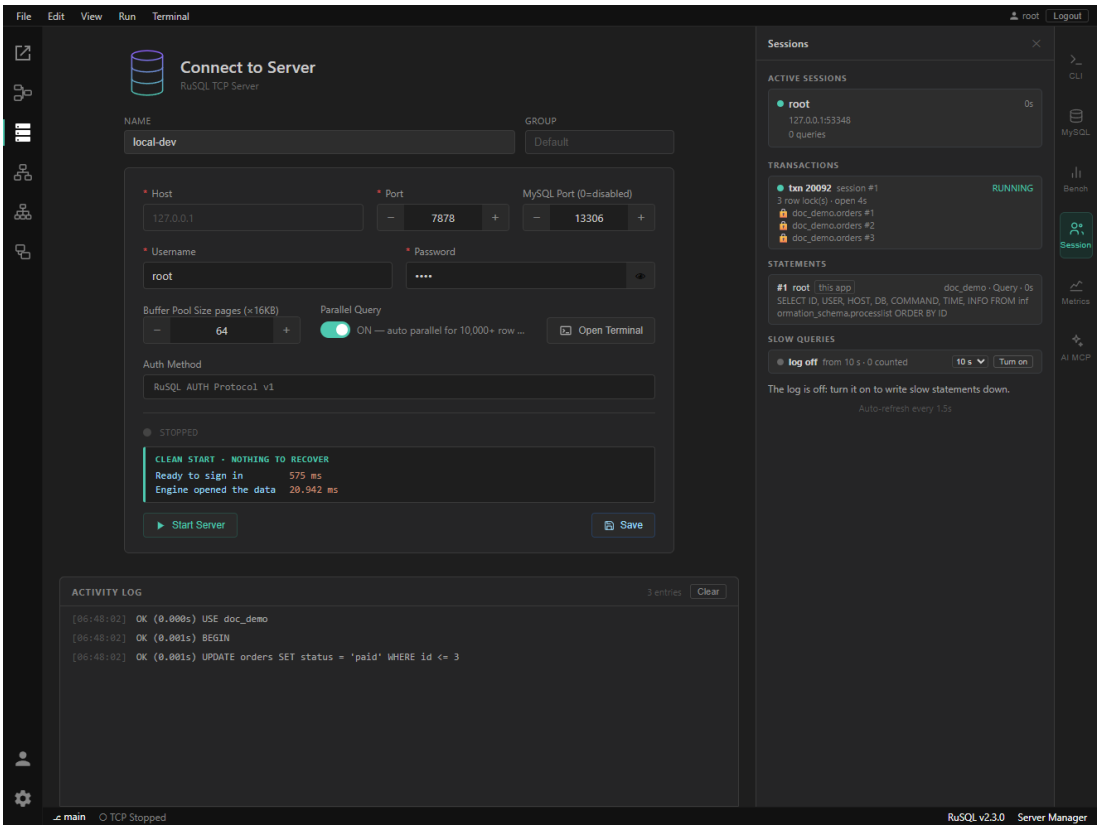


그림 4-6. 세션 · 트랜잭션 · 락 패널

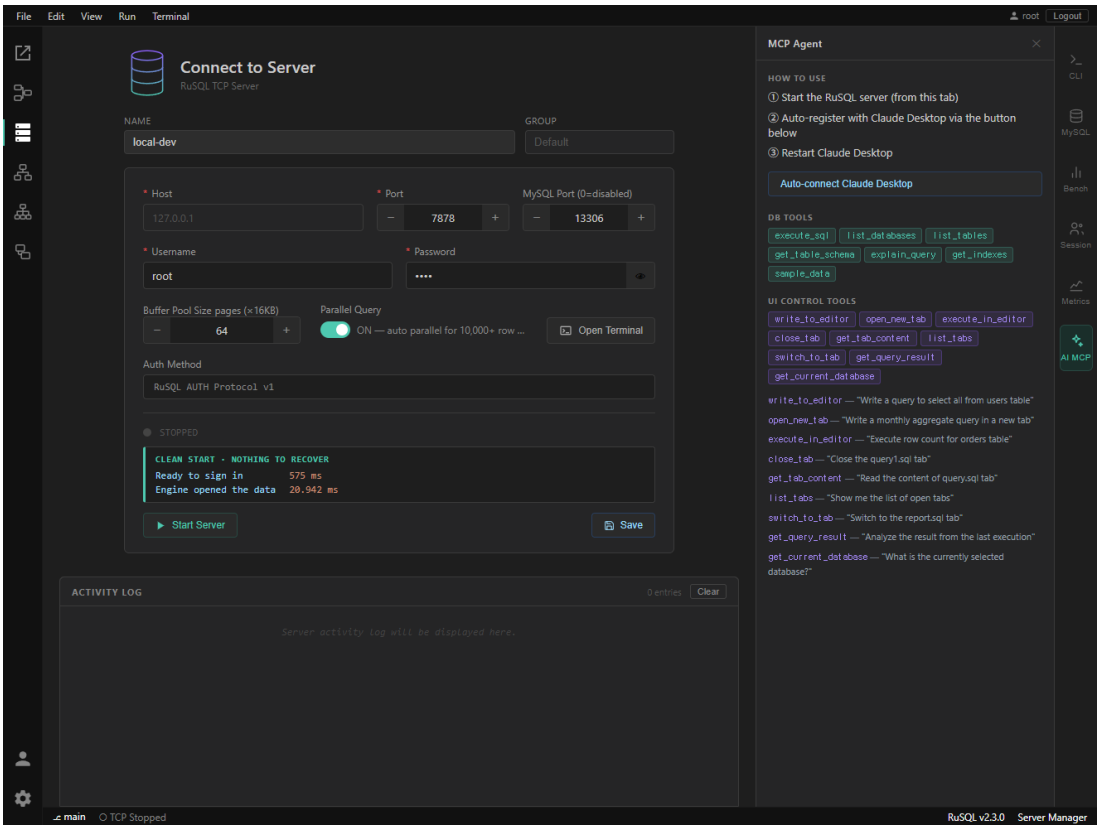


그림 4-7. AI MCP 패널

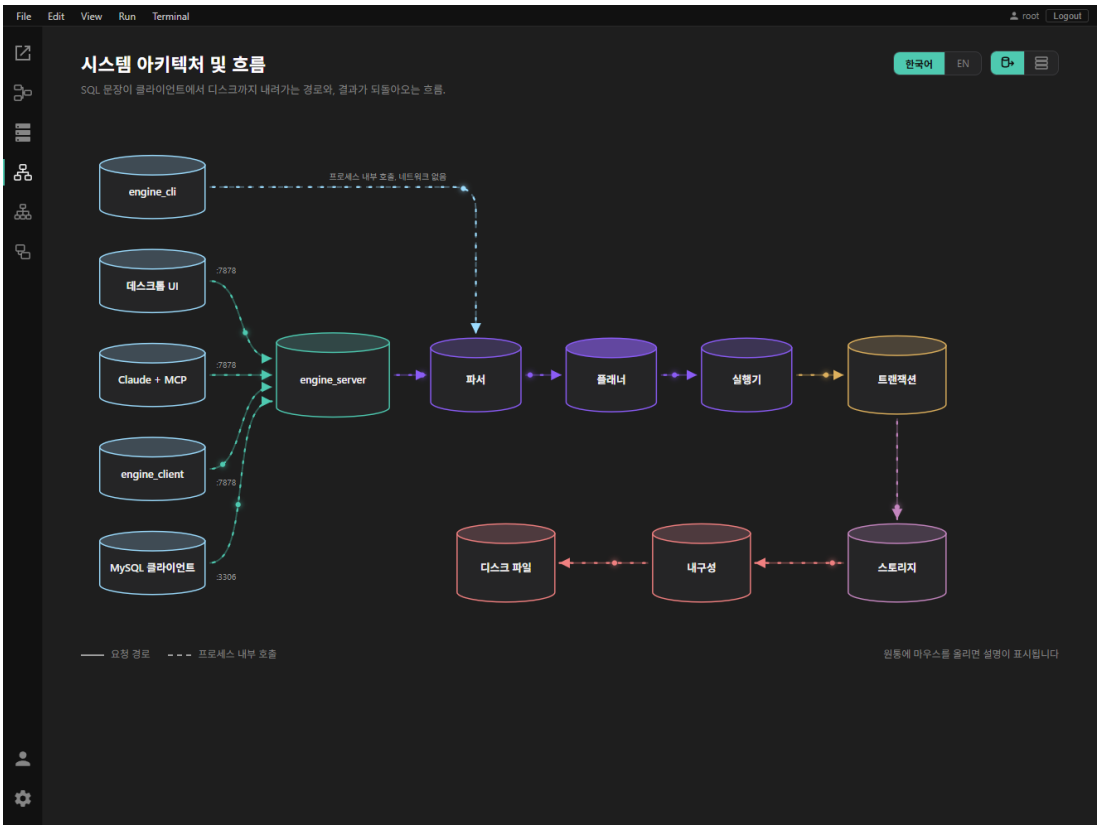


그림 4-8. Diagram 페이지

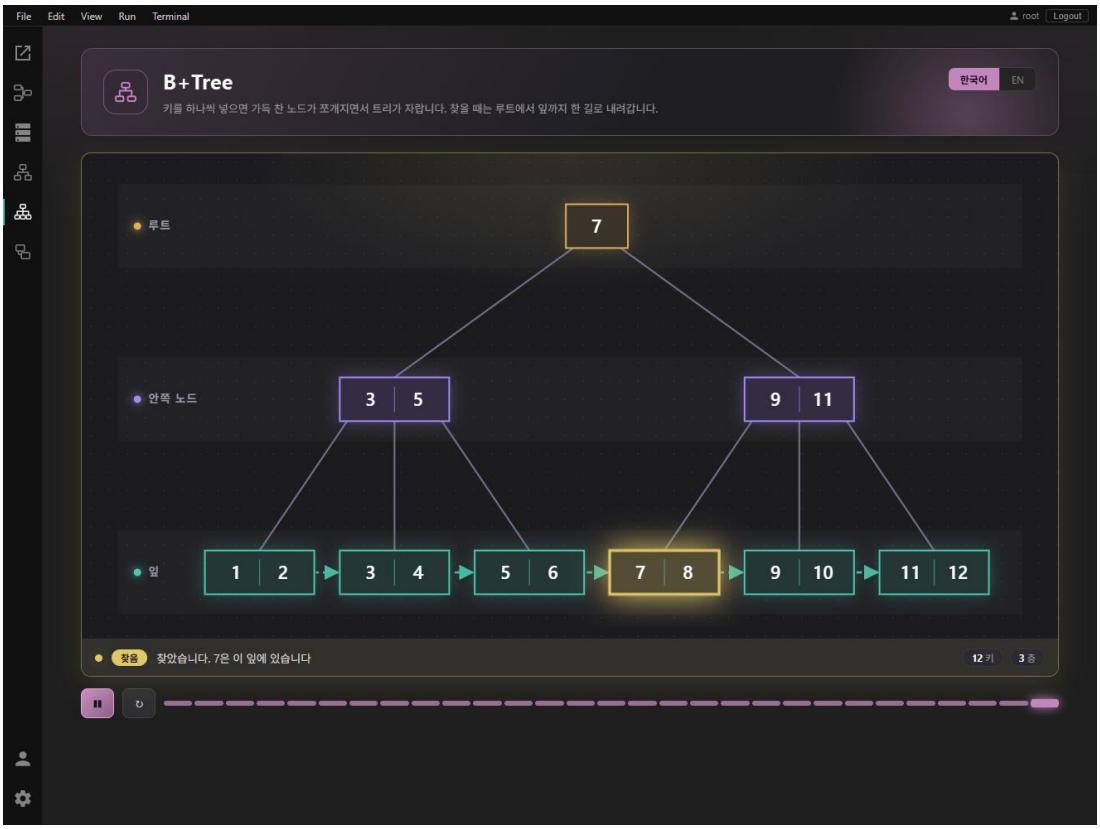


그림 4-9. B+Tree 페이지

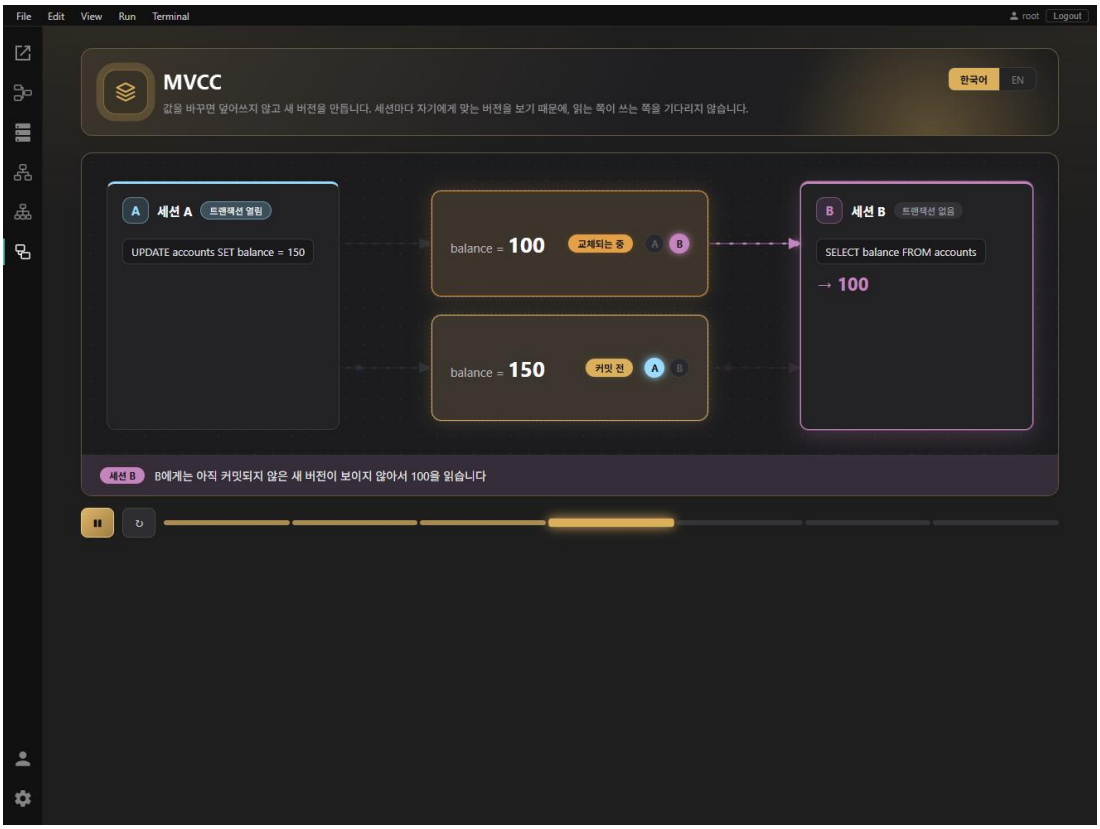


그림 4-10. MVCC 페이지

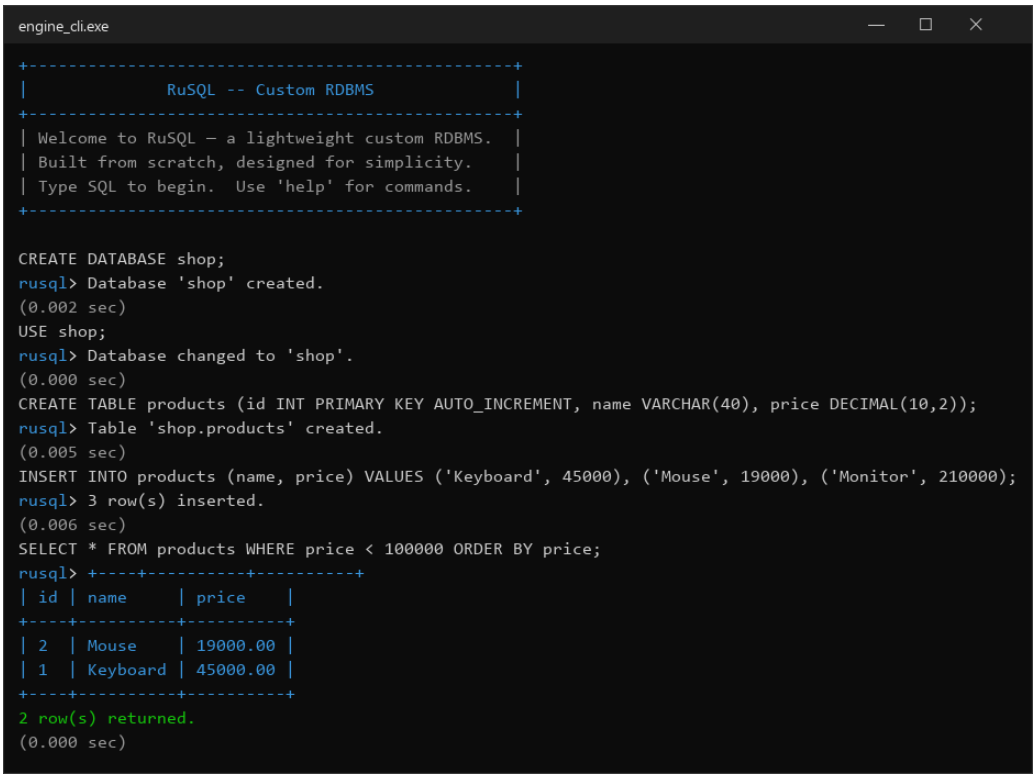


그림 4-11. 터미널 클라이언트 실행 화면

5. 기능적 모델링: Use-Case Diagram

Functional Modeling: Use-Case Diagram

5.1 Use-Case Diagram

그림 5-1은 RuSQL의 액터별 주요 사용사례를 나타낸다.

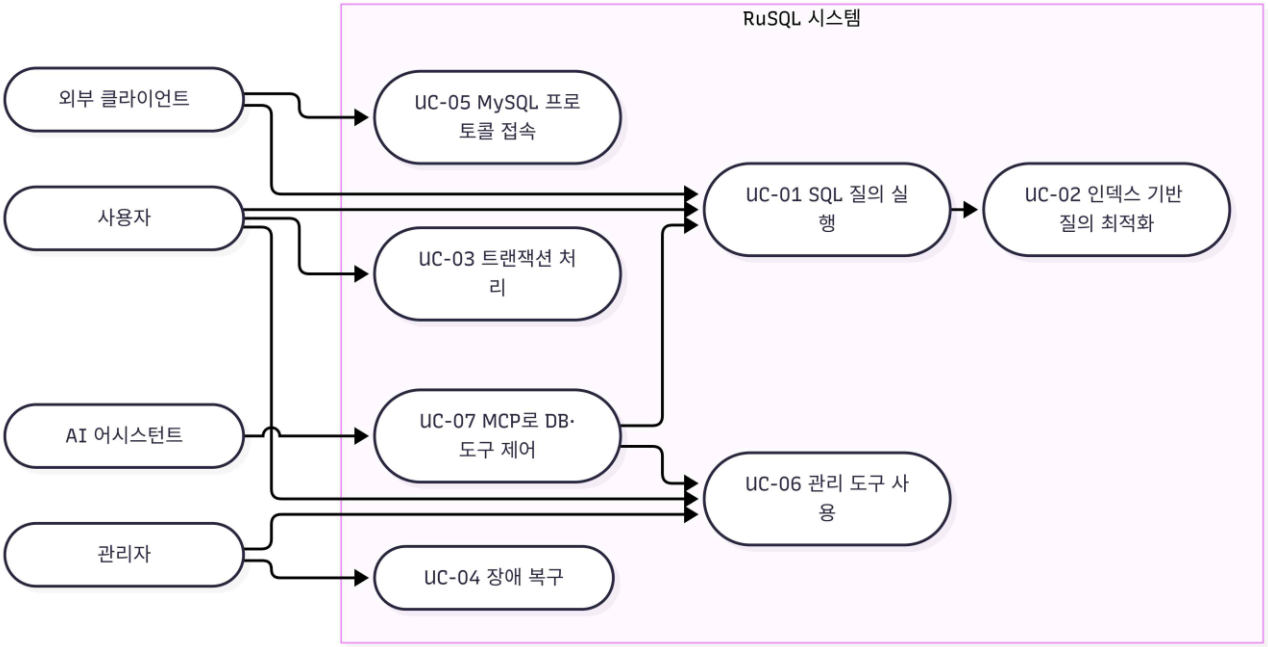


그림 5-1. RuSQL 사용사례 다이어그램

5.2 액터 정의

표 5-1은 RuSQL의 액터와 역할을 정의한다.

표 5-1. 액터 목록

액터명	역할	참고
사용자(User)	SQL을 실행하고 데스크톱 관리 도구로 데이터베이스를 조작하는 학생·개발자	
외부 클라이언트(External Client)	MySQL 프로토콜로 서버에 접속하는 mysql CLI, DBeaver 등	
AI 어시스턴트(AI Assistant)	MCP 도구로 서버와 관리 도구를 자연어로 조작하는 Claude Desktop 등	
관리자(Administrator)	서버를 기동·정지하고 장애 발생 시 복구를 확인하는 운영 담당자	본 프로젝트에서는 사용자가 겸임

5.3 유스케이스 목록

표 5-2는 그림 5-1의 각 사용사례를 설명한다.

표 5-2. 사용사례 목록

UC ID	사용사례 명	관련 액터	설명
UC-01	SQL 질의 실행	사용자, 외부 클라이언트	SQL 문을 파싱하고 실행 계획을 세워 실행한 뒤 결과를 반환한다.
UC-02	인덱스 기반 질의 최적화	사용자	B+Tree·Hash 인덱스와 비용 기반 옵티마이저로 UC-01의 실행 계획을 선택한다.
UC-03	트랜잭션 처리	사용자	BEGIN/COMMIT/ROLLBACK과 행 단위 MVCC, 4단계 격리 수준으로 동시 접속을 처리한다.
UC-04	장애 복구	관리자	서버 재기동 시 WAL·Undo Log로 커밋된 변경을 재현하고 미완료 변경을 되돌린다.
UC-05	MySQL 프로토콜 접속	외부 클라이언트	기존 MySQL 클라이언트가 TCP로 접속하여 SQL을 수행한다.
UC-06	관리 도구 사용	사용자, 관리자	데스크톱 관리 도구에서 SQL 편집·실행, 스키마·ERD 조회, 서버 상태 모니터링을 수행한다.
UC-07	MCP로 DB·도구 제어	AI 어시스턴트	MCP 도구로 자연어 요청을 SQL 실행 또는 관리 도구 화면 조작으로 변환하여 수행한다.

6. 구조적 모델링: Class Diagram

Structural Modeling: Class Diagram

구조적 모델링은 엔진 라이브러리와 서버의 클래스 구조와 패키지 구성을 나타낸다. 6.1절은 클래스 사이의 관계를, 6.2절은 패키지(컴포넌트) 구성을 설명한다.

6.1 Class Diagram

그림 6-1은 파서, 엔진, 카탈로그 · 저장소, 트랜잭션, 서버 계층의 주요 클래스와 그 관계를 나타낸다.

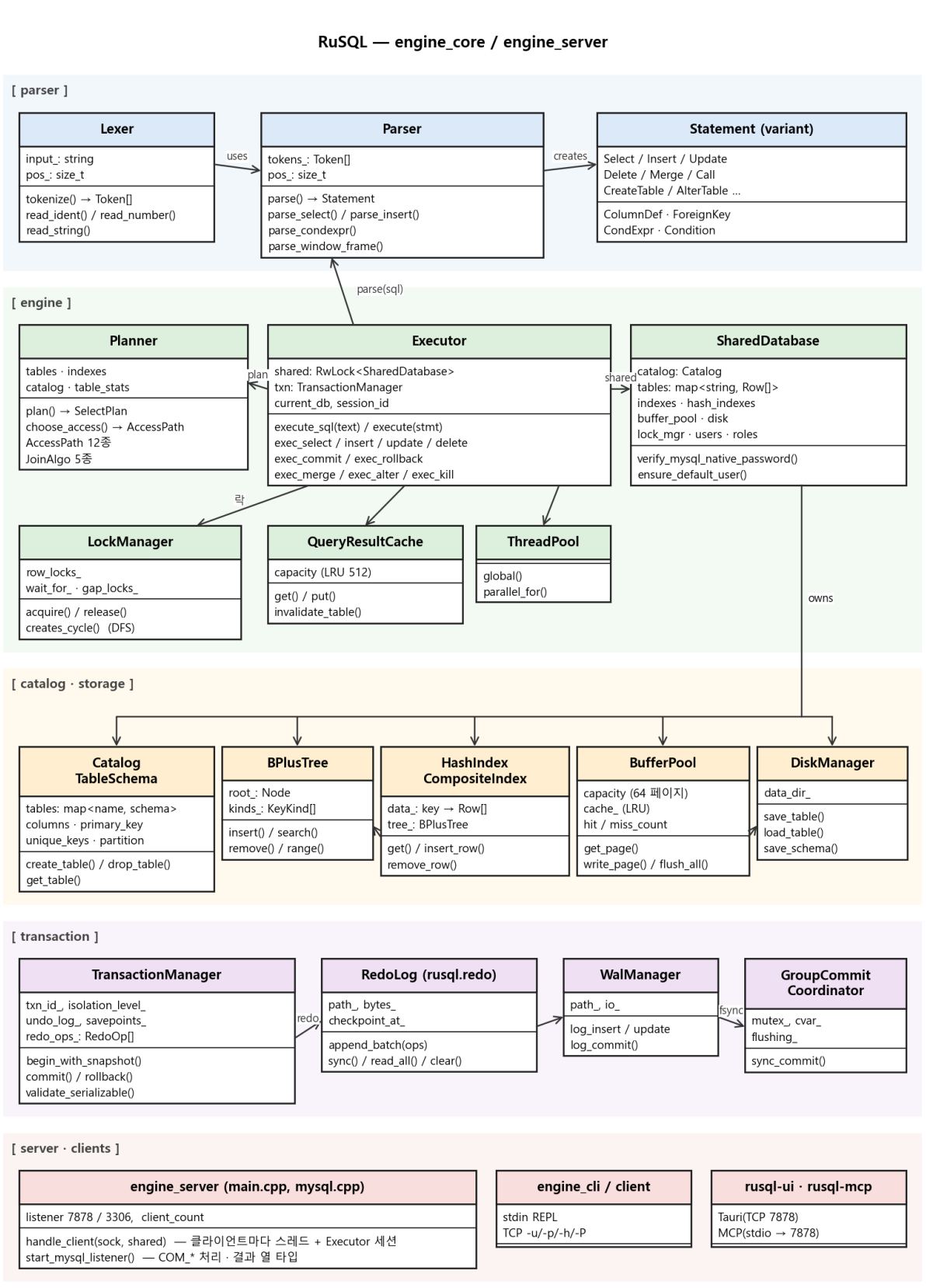
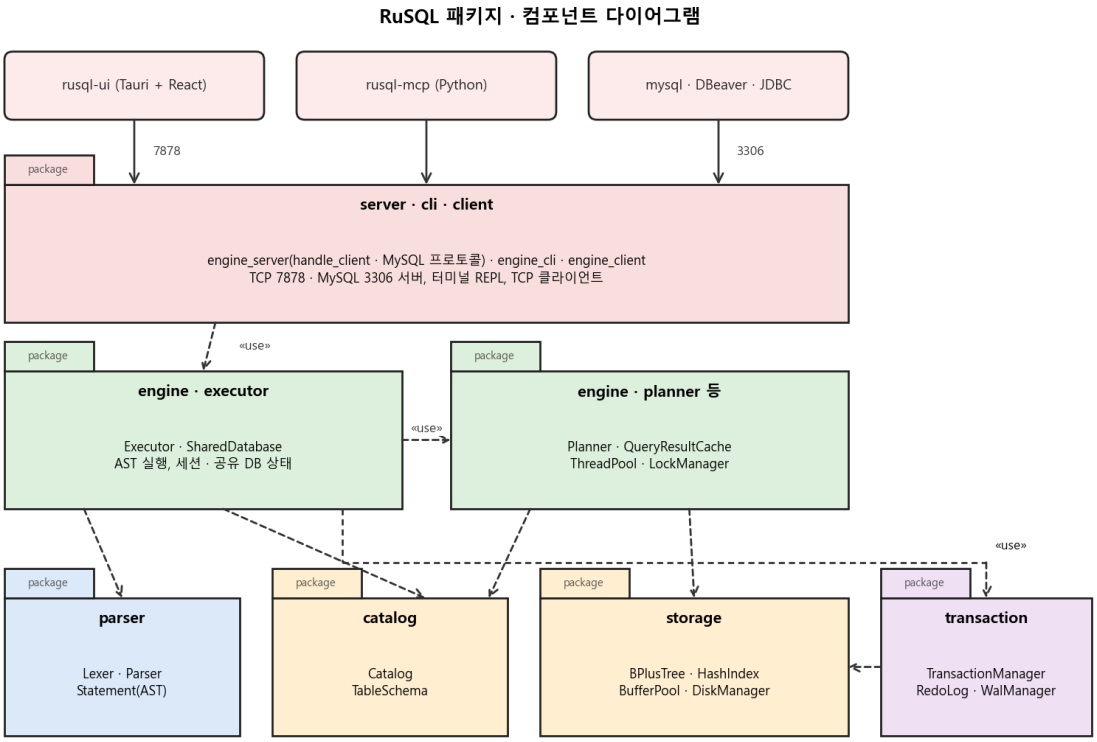


그림 6-1. RuSQL Class Diagram (엔진 라이브러리와 서버)

6.2 패키지 구성

그림 6-2는 패키지(컴포넌트) 사이의 의존 관계를, 표 6-1은 패키지별 위치와 주요 클래스, 역할을 나타낸다.



점선 화살표 «use»는 의존(호출) 방향을 나타낸다. 계층은 위에서 아래로 server → engine → parser · catalog · storage · transaction 순이다.

그림 6-2. 패키지 · 컴포넌트 다이어그램

표 6-1. 패키지 구성

패키지	위치	주요 클래스	역할
parser	core/src/parser	Lexer · Parser · Statement(AST) · ColumnDef · ForeignKey · CondExpr	SQL 문장을 토큰 → AST로 변환
engine · executor	core/src/engine/executor	Executor · SharedDatabase	AST 실행 (DDL · DML · 트랜잭션 · 사용자 · 프로시저 · 트리거 · 운영 문장), 세션 상태와 공유 DB 상태
engine · planner 등	core/src/engine	Planner · QueryResultCache · ThreadPool · LockManager	비용 기반 계획 선택, 결과 캐시, 병렬 실행, 행 단위 락 · 데드락 감지
catalog	core/src/catalog	Catalog · TableSchema	표 · 열 · 제약 · 파티션 메타데이터
storage	core/src/storage	BPlusTree · HashIndex · CompositeIndex · BufferPool · DiskManager	인덱스, 페이지 캐시, .rdb · JSON 파일 입출력
transaction	core/src/transaction	TransactionManager · RedoLog · WalManager · GroupCommitCoordinator	MVCC 트랜잭션 · redo 로그 · 그룹 커밋 · 크래시 복구
server · cli · client	server/src, cli/src, client/src	engine_server(handle_client · MySQL 프로토콜) · engine_cli · engine_client	TCP 7878 / MySQL 3306 서버, 터미널 REPL, TCP 클라이언트

7. 클래스 정의

Class Definitions

클래스 정의는 6장의 클래스 다이어그램에 나타낸 클래스를 계층별로 나누어 각 클래스의 속성과 메서드를 정리한다.

7.1 파서 레이어

표 7-1은 파서 레이어의 Lexer, Parser, Statement 등의 속성과 메서드를 정리한다.

표 7-1. 파서 레이어 클래스

클래스명	속성	메서드
Lexer	input_: string pos_: size_t	tokenize() → vector<Token> read_ident() / read_number() / read_string() / read_quoted_ident() skip_whitespace()
Parser	tokens_: vector<Token> source_: string pos_: size_t enclosing_from_ (서브쿼리 바깥 FROM 이름) window_defs_ (이름 있는 WINDOW)	parse() → Statement (실패 시 ParseError) parse_select / parse_insert / parse_update / parse_delete / parse_create / parse_alter / parse_merge() parse_condexpr → parse_or_expr → parse_and_expr → parse_not_expr parse_value_expr() / parse_window_frame() / parse_over_clause() parse_proc_stmt() / parse_create_trigger() peek() / advance() / expect_ident()
Statement (variant)	Select / Insert / InsertSelect / Update / Delete CreateTable / CreateTableLike / CreateTableAs / AlterTable(AlterAction) Begin / Commit / Rollback / Merge / Call … 수십 개 변형을 std::variant로 보관	—
IsolationLevel (enum)	ReadUncommitted ReadCommitted (기본값) RepeatableRead Serializable	—
ColumnDef ForeignKey UniqueKey	ColumnDef: name, data_type: DataType, primary_key / not_null / unique / auto_increment / default_value / check_expr, on_update_now, is_unsigned, foreign_keys: vector<ForeignKey> ForeignKey: column, ref_table / ref_column, on_delete / on_update: FkAction (Restrict · Cascade · SetNull · SetDefault), name	—

클래스명	속성	메서드
	UniqueKey: name, columns	
CondExpr Condition	CondExpr: variant<And, Or, Not, Leaf> Condition: left: ArithExpr, op: Operator, value: ConditionValue, quantifier (ANY · ALL), left_class / right_class: ValueClass	—

7.2 엔진 · 실행 레이어

표 7-2는 엔진 · 실행 레이어의 Executor, SharedDatabase, Planner 등의 속성과 메서드를 정리한다.

표 7-2. 엔진 · 실행 레이어 클래스

클래스명	속성	메서드
Executor	shared: shared_ptr<RwLock<SharedDatabase>> txn: TransactionManager current_db: string user_vars / proc_vars / prepared_stmts session_id, auth_user, lock_wait_timeout_ms (50000) last_insert_id_, row_count_, found_rows_	execute(Statement) → Result<string, string> execute_sql(text) (Lexer → Parser → execute) exec_select / exec_insert / exec_update / exec_delete / exec_merge() exec_begin / exec_commit / exec_rollback / exec_savepoint() exec_create / exec_alter / exec_create_index / exec_create_view() exec_explain / exec_explain_analyze / exec_suggest_indexes() exec_kill / exec_show_status / exec_call_procedure() recover_from_redo() / maybe_auto_checkpoint() / new_session()
SharedDatabase	catalog: Catalog tables: map<string, vector<Row>> indexes · composite_indexes · hash_indexes views · procedures · triggers · user_functions · synonyms buffer_pool: BufferPool, disk: DiskManager, lock_mgr: LockManager users · grants · roles · role_grants group_commit_coord · table_stats · process_list · counters · slow_log	verify_mysql_native_password() ensure_default_user() (shared_ptr<RwLock<SharedDatabase>>로 세션 사이에 전체 DB 상태 공유)
Planner	tables · indexes · composite_indexes · hash_indexes · catalog · table_stats 참조 SelectPlan { base: TablePlan, joins: vector<JoinPlan> }	plan(table, condition, joins) → SelectPlan plan_covering() / choose_access(table, cond, pk) → AccessPath AccessPath 12종: SeqScan / PkPoint /

클래스명	속성	메서드
		PkBetween / PkRange / SecondaryPoint / SecondaryRange / SecondaryBetween / CompositeIndexPath / CompositeIndexPrefix / HashPoint / SecondaryLikePrefix / IndexIntersection JoinAlgo 5종: NestedLoop / Hash / SortMerge / IndexNL / ReverseIndexNL estimate_rows() / estimate_cost()
QueryResultCache	capacity (LRU 512) entries_, table_to_keys_, table_generation_	get() / put() invalidate_table() / clear() / len()
ThreadPool	프로세스 전역 스레드 풀	global() parallel_for(n, fn) — WHERE 필터 · GROUP BY · 정렬 · 집계의 병렬 실행
ServerCounters SlowLog ProcessInfo	ServerCounters: 문장 종류별 · 연결 · 락 대기 카운터 SlowLog: enabled, threshold_us, entries_ (최대 1000), file_path_ ProcessInfo: 세션별 사용자 · DB · 실행 중인 문장 · 시작 시각	SlowLog: record() / entries() / append_to_file() (SHOW STATUS · SHOW PROCESSLIST · KILL · information_schema.slow_log의 바탕)

7.3 카탈로그 · 저장소 레이어

표 7-3은 카탈로그 · 저장소 레이어의 Catalog, BPlusTree, HashIndex, BufferPool, DiskManager 등의 속성과 메서드를 정리한다.

표 7-3. 카탈로그 · 저장소 레이어 클래스

클래스명	속성	메서드
Catalog TableSchema	Catalog: tables: map<string, TableSchema> TableSchema: name, columns: vector<ColumnDef>, primary_key_columns, check_constraints, unique_keys, auto_increment_counters, partition_info	Catalog: create_table / create_table_full / drop_table() get_table() / get_table_mut() TableSchema: identity_columns()
BPlusTree	root_: unique_ptr<Node> (InternalNode / LeafNode) kinds_: vector<KeyKind> (숫자 · 글자 키 구분) mutex_	insert(key, value) / search(key) / remove(key) range_search() / scan_from() / scan_to() / range_keys() all_values() / collect_all_kv() / len()
HashIndex	table, column data_: map<string, vector<Row>>	rebuild() / get(key) insert_row() / remove_row()

클래스명	속성	메서드
		bucket_count() / row_count()
CompositeIndex	table, columns, pk_columns tree_: BPlusTree	insert_row() / remove_row() search_exact() / rows_with_prefix() / lookup() rebuild()
BufferPool	capacity (기본 64 페이지) cache_: map<table, (Page, tick)> (LRU) hit_count / miss_count	get_page(table, disk) → rows write_page() / write_through() / flush_all() / invalidate() hit_rate() / usage()
DiskManager	data_dir_: string (표 .rdb · 스키마 JSON · data/_system 전역 JSON 경로)	save_table / load_table() · save_schema / load_schema() save_* / load_* : index_meta · views · users · grants · roles · synonyms · procedures · triggers · functions create_db_dir / drop_db_dir / list_databases()

7.4 트랜잭션 · 동시성 레이어

표 7-4는 트랜잭션 · 동시성 레이어의 TransactionManager, RedoLog, WalManager, GroupCommitCoordinator, LockManager의 속성과 메서드를 정리한다.

표 7-4. 트랜잭션 · 동시성 레이어 클래스

클래스명	속성	메서드
TransactionManager	active_, txn_id_ undo_log_: vector<UndoEntry> wal_: WalManager isolation_level_, frozen_ctx_ (스냅샷) read_set_ (SERIALIZABLE 검증) savepoints_ redo_ops_: vector<RedoOp>	begin_with_snapshot() / commit() / rollback() / abort() create_savepoint() / rollback_to_savepoint() / release_savepoint() record_read() / validate_serializable() take_redo_ops() / do_checkpoint()
RedoLog RedoOp	RedoLog: path_ (rusql.redo), bytes_, checkpoint_at_, appended_ / synced_ RedoOp: kind (InsertVersion · SetXmax · TableFlushed), table, row_json, xmax	append_batch(ops) / sync() read_all() / clear() bytes() / checkpoint_at()
WalManager	path_: string io_: shared_ptr<TxnIoShared>	log_insert / log_update / log_delete() log_commit / log_rollback / log_checkpoint() read_all() / clear() / remove_txn() file_size() / needs_auto_checkpoint()
GroupCommitCoordi	mutex_, cvar_	sync_commit()

클래스명	속성	메서드
nator	flushing_, generation_	Leader: 단일 fsync 수행 Follower: 조건 변수 대기 후 완료
LockManager	row_locks_: map<(table, pk), LockEntry> wait_for_: map<txn, txn> deadlock_history_ gap_locks_, predicate_reads_	acquire(table, pk, txn_id) → LockResult acquire_shared() / release(txn_id) acquire_gap() / register_predicate_read() creates_cycle(from, to) → bool (DFS) waits_total() / holder()

7.5 서버 · 운영

표 7-5는 서버 · 운영 레이어의 engine_server와 KillScope 등의 속성과 메서드를 정리한다.

표 7-5. 서버 · 운영 레이어 클래스

클래스명	속성	메서드
engine_server (main.cpp, mysql.cpp)	listener: 7878 (커스텀) / 3306 (MySQL) client_count shared: shared_ptr<RwLock<SharedDatabase>>	handle_client(sock, shared, …) — 클라이언트마다 스레드 + Executor::new_session() start_mysql_listener() — 핸드셰이크 · 인증 · COM_* 처리 · 결과 열 타입 전달 wire_error() — 오류 메시지에서 MySQL 오류 번호 · SQLSTATE 결정
KillScope ReadPhase Watch · ParallelStop	문장마다 KillScope(중단 요청 · 실행 시간 제한) ReadPhase: 읽기 단계 표시	tick() / check_now() — 반복문에서 KILL · 시간 제한 확인 ParallelStop — 병렬 작업자가 정지 신호를 보고 되돌아옴

8. 주요 Use-Case 시퀀스 다이어그램

Sequence Diagram for Major Use-Cases

주요 사용사례의 처리 흐름을 시퀀스 다이어그램으로 나타낸다. 각 절은 그림으로 전체 흐름을 보이고 표로 단계별 메시지와 설명을 정리하며, 5장의 UC-01부터 UC-07까지를 모두 다룬다.

8.1 UC-01: SQL 질의 실행 시퀀스

그림 8-1은 UC-01(SQL 질의 실행)의 메시지 흐름을, 표 8-1은 단계별 설명을 나타낸다.

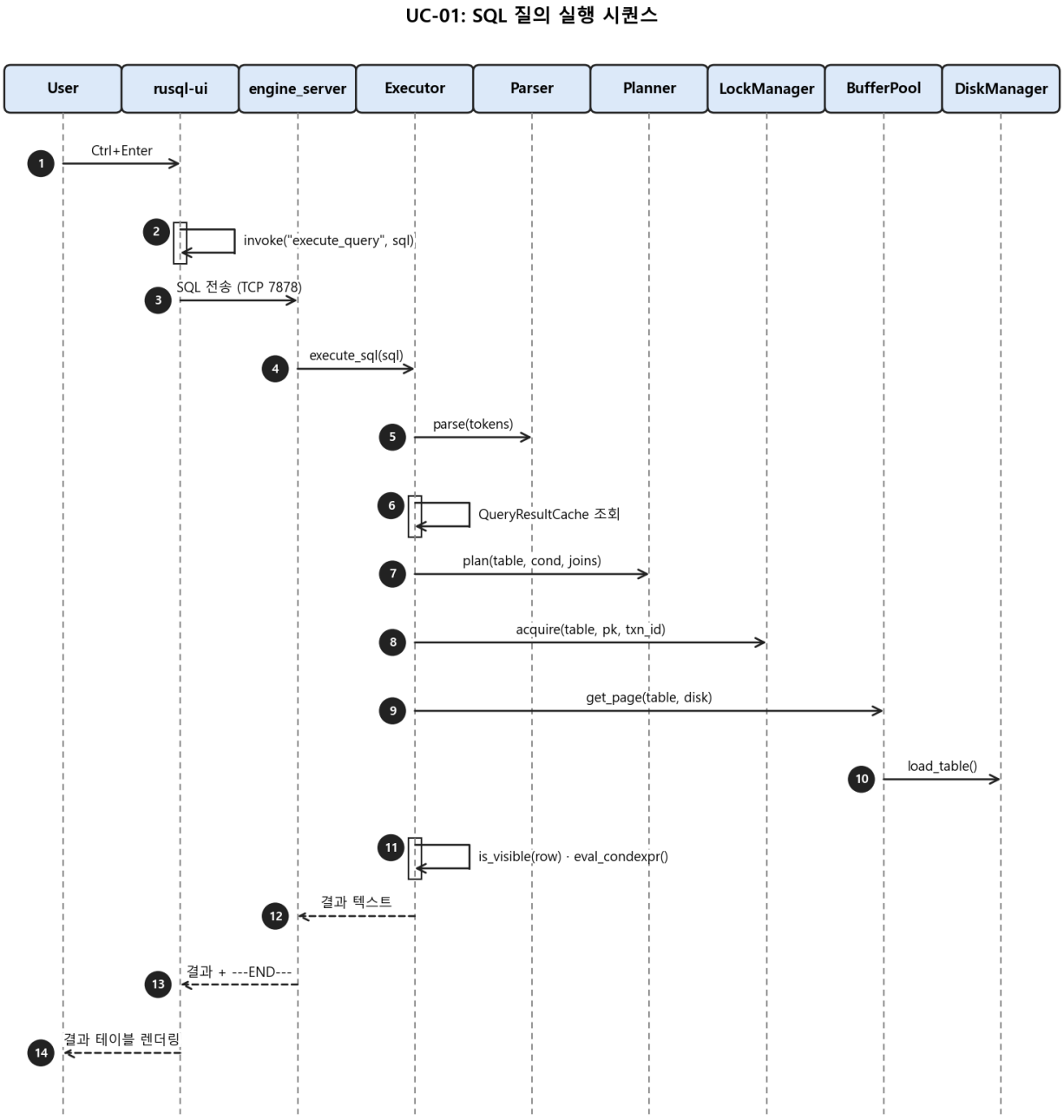


그림 8-1. UC-01 SQL 질의 실행 시퀀스

표 8-1. UC-01 SQL 질의 실행 단계별 설명

단계	참여자	메시지	설명
1	User	Ctrl+Enter 입력	쿼리 실행 트리거
2	rusql-ui	invoke("execute_query", sql)	Tauri 커맨드 호출, 문장을 ;로 나눠 하나씩 전송
3	rusql-ui → engine_server	TCP 7878로 SQL 전송	AUTH가 끝난 연결 사용, 응답은 ---END---로 끝남
4	engine_server → Executor	execute_sql(sql)	클라이언트 스레드의 세션 Executor 호출
5	Executor → Parser	Lexer 토큰화 → parse(tokens)	SQL → 토큰 리스트 → AST (Statement)
6	Executor	QueryResultCache 조회	같은 문장 · 같은 표 상태면 캐시 사용, 표가 바뀌면 무효화
7	Executor → Planner	plan(table, condition, joins)	AccessPath · 조인 알고리즘 · 조인 순서 선택
8	Executor → LockManager	acquire(table, pk, txn_id)	FOR UPDATE · 쓰기 시 행 잠금 (충돌하면 대기, 데드락 검사)
9	Executor → BufferPool	get_page(table, disk)	캐시에서 행 조회
10	BufferPool → DiskManager	load_table(table)	(캐시 미스 시) .rdb 읽기 · LZ4 해제
11	Executor	is_visible(row) · eval_condexpr()	MVCC 가시성 필터와 WHERE 평가
12	Executor → engine_server	결과 텍스트 (열 · 행 · 행 수)	포맷된 결과 반환
13	engine_server → rusql-ui	결과 + "---END---" 전송	TCP 소켓으로 결과 전달
14	rusql-ui → User	결과 테이블 렌더링	화면 출력

8.2 UC-02: 인덱스 기반 질의 최적화 시퀀스

그림 8-2는 UC-02(인덱스 기반 질의 최적화)의 메시지 흐름을, 표 8-2는 단계별 설명을 나타낸다.

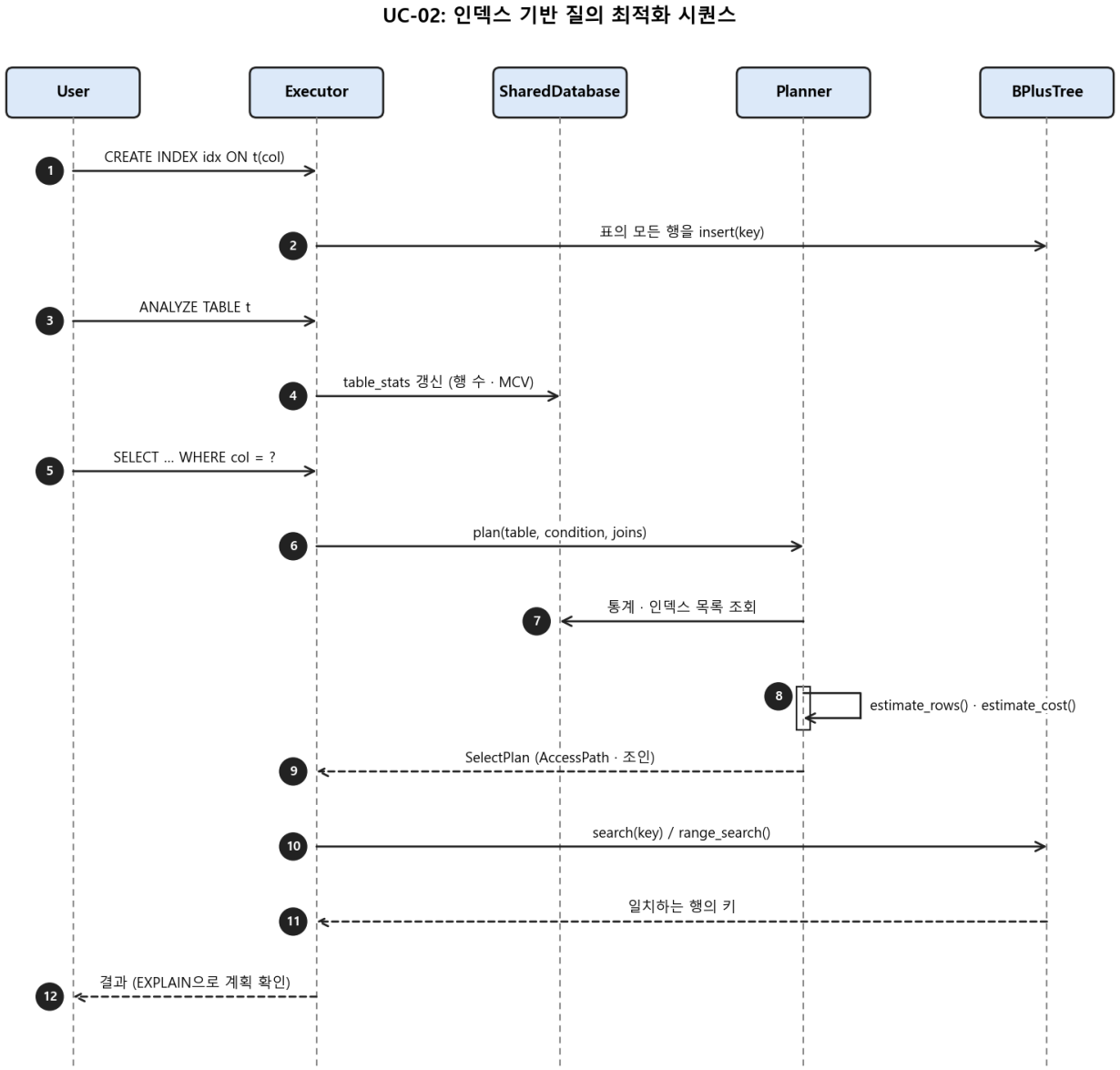


그림 8-2. UC-02 인덱스 기반 질의 최적화 시퀀스

표 8-2. UC-02 인덱스 기반 질의 최적화 단계별 설명

단계	참여자	메시지	설명
1	User → Executor	CREATE INDEX idx ON t(col)	인덱스 생성 요청
2	Executor → BPlusTree	표의 모든 행을 insert(key)	저장된 행으로 B+Tree 인덱스를 만든다
3	User → Executor	ANALYZE TABLE t	통계 수집 요청
4	Executor → SharedDatabase	table_stats 갱신	행 수 · 서로 다른 값 수 · MCV · 히스토그램을 저장
5	User → Executor	SELECT … WHERE col = ?	조건이 있는 질의 실행

단계	참여자	메시지	설명
6	Executor → Planner	plan(table, condition, joins)	실행 계획 요청
7	Planner → SharedDatabase	통계 · 인덱스 목록 조회	사용할 수 있는 인덱스와 통계를 읽음
8	Planner	estimate_rows() · estimate_cost()	후보 AccessPath(SeqScan, 인덱스 조회 등)별 행 수와 비용을 추정해 가장 낮은 것을 고름
9	Planner → Executor	SelectPlan (AccessPath · 조인)	접근 경로 · 조인 알고리즘 · 조인 순서가 담긴 계획 반환
10	Executor → BPlusTree	search(key) / range_search()	선택한 접근 경로로 인덱스를 조회 (전체 스캔 대신)
11	BPlusTree → Executor	일치하는 행의 키	키로 행을 읽어 MVCC 가시성과 나머지 조건을 확인
12	Executor → User	결과	EXPLAIN으로 선택된 계획(접근 경로 · 예상 비용)을, SUGGEST INDEX로 인덱스 제안을 확인할 수 있음

8.3 UC-03: 트랜잭션 처리 시퀀스 (redo 로그)

그림 8-3은 UC-03(트랜잭션 처리)의 메시지 흐름을, 표 8-3은 단계별 설명을 나타낸다.

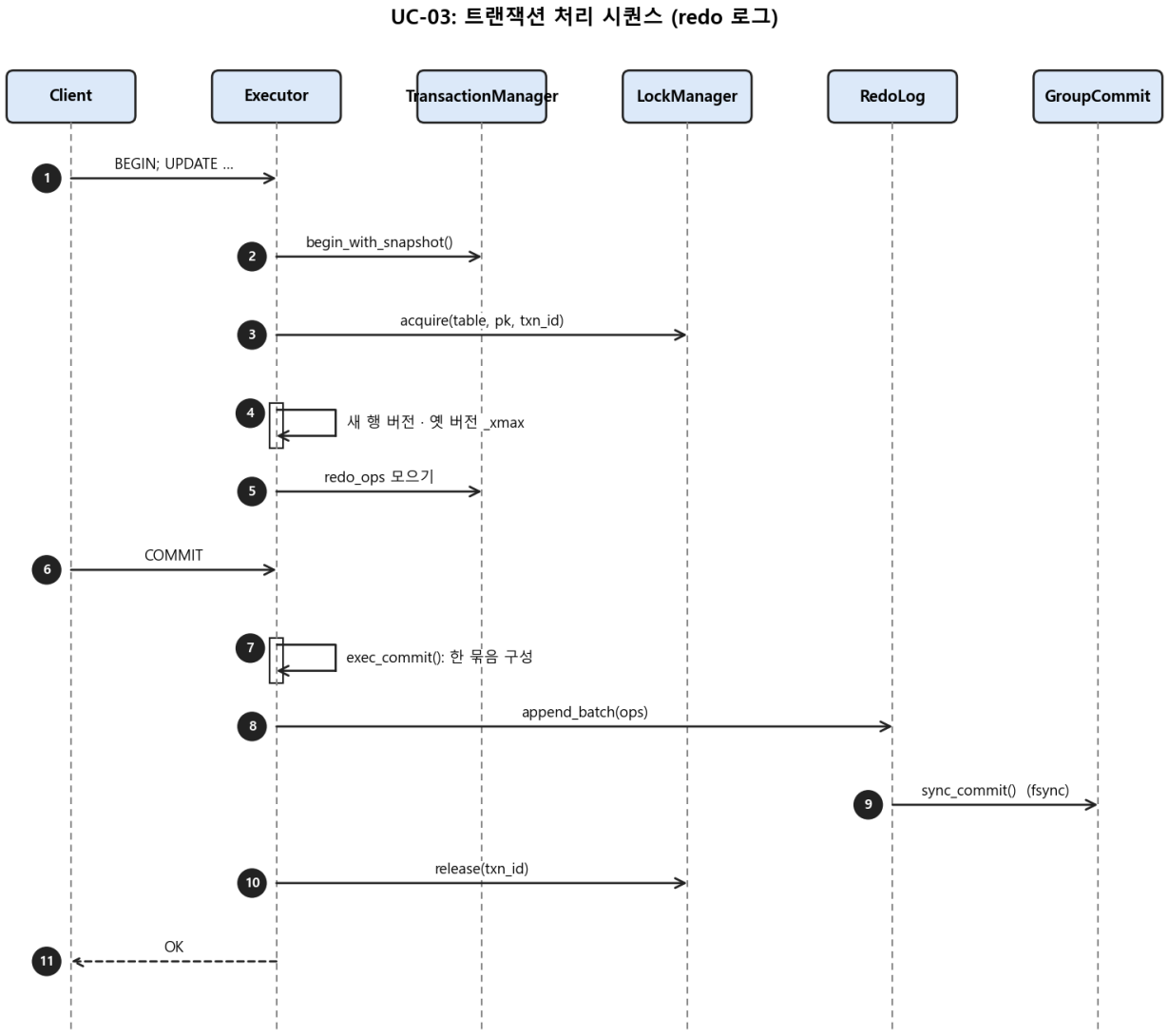


그림 8-3. UC-03 트랜잭션 처리 시퀀스

표 8-3. UC-03 트랜잭션 처리 단계별 설명

단계	참여자	메시지	설명
1	Client	BEGIN; UPDATE …	트랜잭션 시작 · 행 변경 요청
2	Executor → TransactionManager	begin_with_snapshot()	txn_id 부여, 격리 수준에 맞는 스냅샷
3	Executor → LockManager	acquire(table, pk, txn_id)	행 락 (충돌하면 대기, 데드락 검사)
4	Executor	새 행 버전 추가 · 옛 버전 _xmax 표시	MVCC 버전 체인 (_xmin = txn_id)
5	Executor → TransactionManager	redo_ops 모으기 (InsertVersion · SetXmax)	변경을 메모리에 모아 둠

단계	참여자	메시지	설명
6	Client	COMMIT	커밋 요청
7	Executor	exec_commit() — RedoOp를 한 묶음으로 구성	체크섬이 붙은 묶음 만들기
8	Executor → RedoLog	append_batch(ops)	rusql.redo에 묶음 추가
9	RedoLog → GroupCommitCoordinator	sync_commit()	여러 세션의 COMMIT을 하나의 fsync로 묶음 (Leader 1회 fsync, Follower 대기)
10	Executor → LockManager	release(txn_id)	락 해제, 기다리던 세션 깨움
11	Executor → Client	OK	fsync 뒤에 응답 — 표 파일은 체크포인트에서만 다시 씀

8.4 UC-04 · UC-05: 서버 기동 · 복구 · 접속 시퀀스

그림 8-4는 UC-04 · UC-05(서버 기동 · 복구 · 접속)의 메시지 흐름을, 표 8-4는 단계별 설명을 나타낸다.

UC-04 · UC-05: 서버 기동 · 복구 · 접속 시퀀스

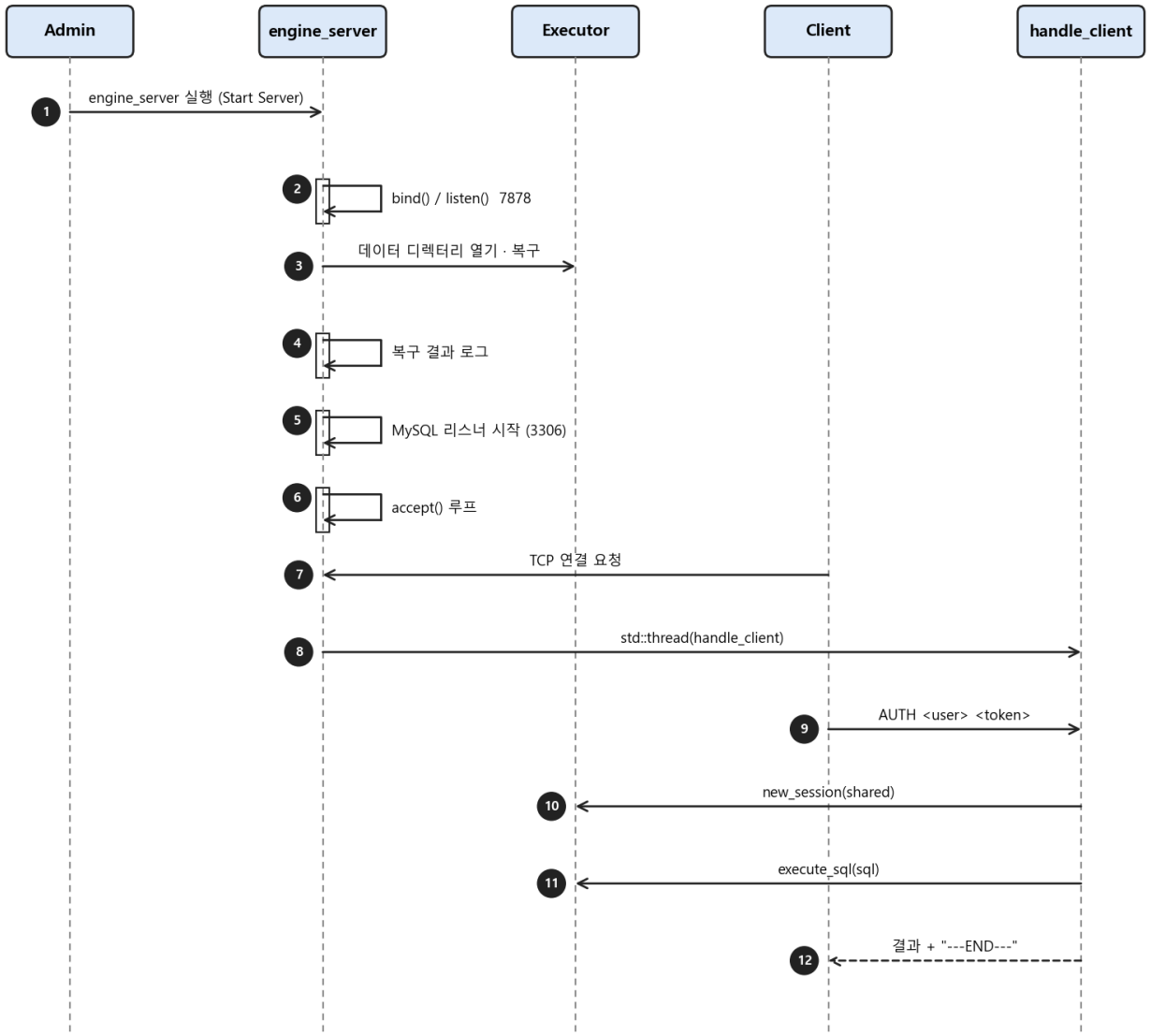


그림 8-4. UC-04 · UC-05 서버 기동 · 복구 · 접속 시퀀스

표 8-4. UC-04 · UC-05 서버 기동 · 복구 · 접속 단계별 설명

단계	참여자	메시지	설명
1	Admin	engine_server 실행 (Server Manager의 Start Server 포함)	서버 프로세스 시작
2	engine_server	bind() / listen() — 127.0.0.1:7878	커스텀 프로토콜 포트 열기
3	engine_server → Executor	데이터 디렉터리 열기 (Executor 부팅)	표 · 스키마 로드, redo 로그 재생(크래시 복구), 인덱스 재생성
4	engine_server	복구 결과 로그 (열린 시간 · 되살린 변경 수)	Server Manager의 복구 카드가 이 기록을 표시

단계	참여자	메시지	설명
5	engine_server	Ctrl+C 핸들러 등록 · start_mysql_listener()	MySQL 프로토콜(3306) 동시 기동
6	engine_server	accept() 루프	클라이언트 접속 대기
7	Client	TCP 연결 요청	클라이언트 접속
8	engine_server	std::thread(handle_client)	클라이언트별 스레드 생성
9	Thread	AUTH <user> <token> 검증	챌린지-응답 확인 후 "OK authenticated" 응답
10	Thread → Executor	Executor::new_session(shared)	세션 Executor 생성 · 접속 목록 등록
11	Thread → Executor	execute_sql(sql) → Result<string>	수신한 쿼리 실행 후 결과 반환
12	Thread → Client	결과 + "---END---\n" 전송	TCP 소켓으로 결과 전달

8.5 UC-06: 관리 도구(ERD Editor) 시퀀스

그림 8-5는 UC-06(관리 도구 사용)의 메시지 흐름을, 표 8-5는 단계별 설명을 나타낸다.

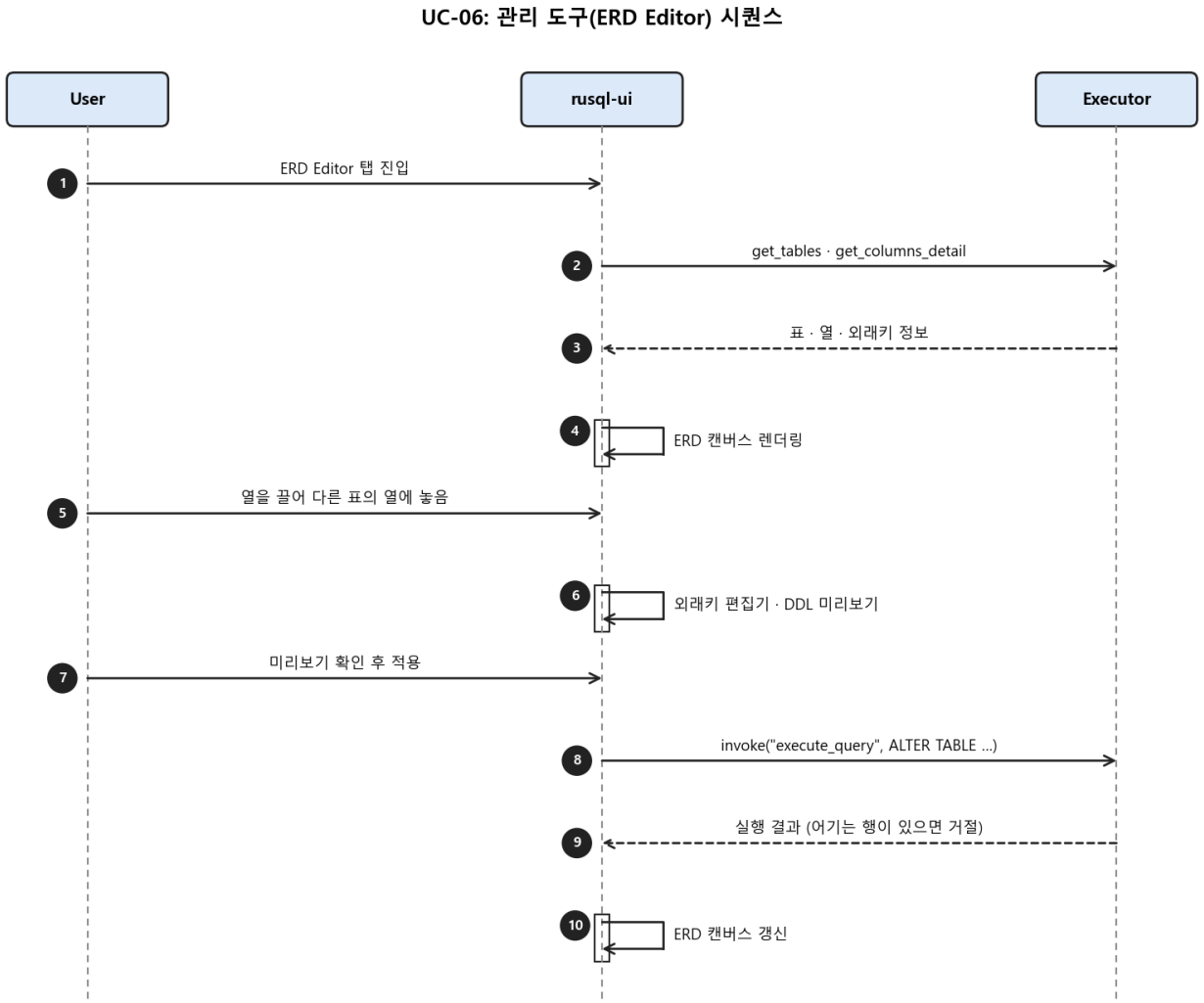


그림 8-5. UC-06 관리 도구(ERD Editor) 시퀀스

표 8-5. UC-06 관리 도구(ERD Editor) 단계별 설명

단계	참여자	메시지	설명
1	User	ERD Editor 탭 진입	rusql-ui ERD 뷰 활성화
2	rusql-ui	invoke("get_tables") · invoke("get_columns_detail")	표 목록과 열 · 외래키 정보 조회
3	Executor → rusql-ui	catalog 정보 반환	표 / 열 / FK 구조 전달
4	rusql-ui	ERD 캔버스 렌더링	표를 카드로, FK를 선으로 시각화
5	User	열을 끌어 다른 표의 열에 놓음	FK 관계 설정 (새 표 · 열 편집도 가능)
6	rusql-ui	외래키 편집기 — DDL 미리보기	UI 조작 결과를 ALTER TABLE … ADD FOREIGN KEY SQL로 변환
7	User	미리보기 확인 후 적용	실행 승인
8	rusql-ui	invoke("execute_query", sql)	생성된 SQL을 서버에 전달
9	Executor → rusql-ui	Result<string>	실행 결과 반환 (이미 어기는 행이 있으면 거절 메시지)
10	rusql-ui	ERD 캔버스 갱신	새 관계 반영

8.6 UC-07: MCP 로 DB · 도구 제어 시퀀스 (Claude Desktop)

그림 8-6은 UC-07(MCP로 DB · 도구 제어)의 메시지 흐름을, 표 8-6은 단계별 설명을 나타낸다.

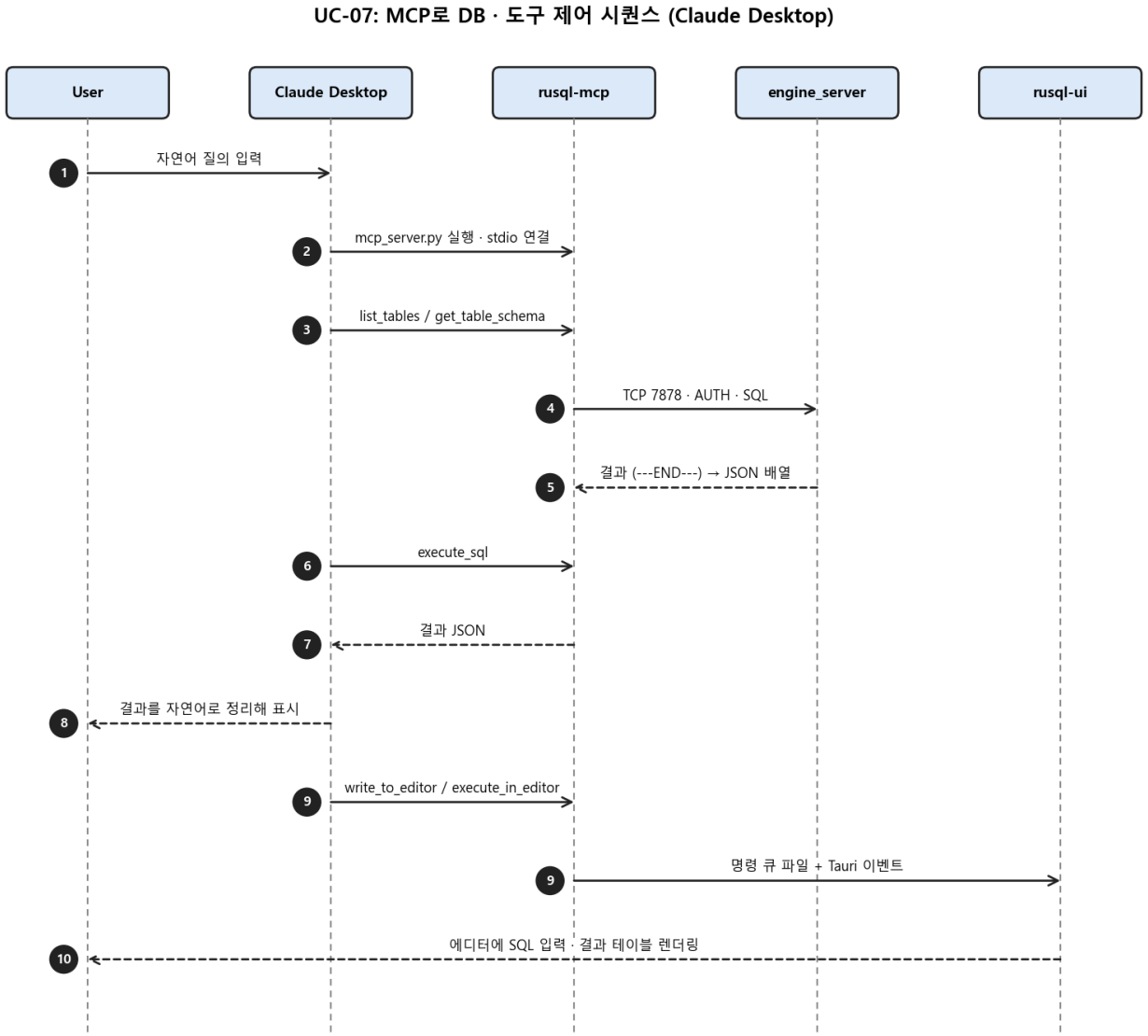


그림 8-6. UC-07 MCP로 DB · 도구 제어 시퀀스

표 8-6. UC-07 MCP로 DB · 도구 제어 단계별 설명

단계	참여자	메시지	설명
1	User	Claude Desktop에 자연어 질의 입력	예: "나이가 25 이상인 사용자 보여줘"
2	Claude Desktop	mcp_server.py 프로세스 spawn	stdio JSON-RPC 연결 수립
3	Claude → rusql-mcp	list_tables / get_table_schema 도구 호출	스키마 정보 수집
4	rusql-mcp → engine_server	TCP 7878 연결 · AUTH · SQL 전송	도구 호출마다 연결 생성 (재시도 포함)
5	engine_server → rusql-mcp	---END--- 구분자로 결과 수신	SELECT 결과를 구조화된 JSON 배열로 변환

단계	참여자	메시지	설명
6	Claude → rusql-mcp	execute_sql 호출	SQL 실행 (위험 SQL은 confirm_dangerous_sql 확인 필요)
7	rusql-mcp → Claude	결과 JSON 반환	실행 결과 전달
8	Claude Desktop → User	결과를 자연어로 정리하여 표시	사용자에게 답변 출력
9	Claude → rusql-mcp	write_to_editor / execute_in_editor (선택)	앱의 쿼리 에디터에 SQL을 넣고 실행 (파일 기반 명령 큐 + Tauri 이벤트)
10	rusql-ui → User	결과 테이블 렌더링	화면 출력